

Opinnäytetyö (AMK)

Tieto- ja viestintätekniikka

2025

Tino Teittinen

Kosketustoimintojen arviointi ja vertailu Kotlin Multiplatformin sekä React Nativen välillä

Opinnäytetyö (AMK) | Tiivistelmä

Turun ammattikorkeakoulu

Tieto- ja viestintätekniikan koulutus

2025 | 69 sivua

Tino Teittinen

Kosketustoimintojen arviointi ja vertailu Kotlin Multiplaformin sekä React Nativen välillä

Opinnäytetyössä tutkittiin kosketustoiminnallisuuden toteuttamista kahdella suositulla kehitysalustalla, Kotlin Multiplatformilla ja React Nativella.

Kosketustoiminnoilla tarkoitetaan eleitä ruudulla, joita käyttäjä tekee sormella tai muulla kosketuksen tunnistavalla välineellä.

Työn tavoitteena oli selvittää, miten kosketustoimintojen käyttöönotto, suoriutuminen ja toiminnallisuus erosivat kehysten välillä. Molemmille alustoille kehitettiin vastaavat sovellustoiminnallisuudet, ja kehitysprosessin aikana tehtiin havaintoja, joiden pohjalta teknologioita vertailtiin sekä osa-alueittain että kokonaisuutena.

Toteutuksissa oli paljon yhtäläisyyksiä ja niiden kompleksiset toiminnot edellyttivät manuaalista kehitystä valmiiden eleentunnistimien rajoitteiden vuoksi. React Native tarjosi lähestyttävämmän kehityskokemuksen ja verkkomateriaalin, kun taas Kotlin Multiplatform sisälsi ominaisuuksia, jotka yksinkertaistivat kehitystyötä.

Työtä voisi jatkaa tutkimalla uusia eleyhdistelmiä sekä selvittämällä miten eri kolmannen osapuolen kirjastot vaikuttaisivat kehitysprosessiin ja voisivatko ne yksinkertaistaa tai automatisoida manuaalisia vaiheita.

Asiasanat:

kosketustoiminnot, React Native, Kotlin, mobiilisovellus

Bachelor's Thesis | Abstract

Turku University of Applied Sciences

Bachelor of Engineering, Information and Communications Technology

2025 | 69 pages

Tino Teittinen

Evaluation and Comparison of Touch Functionalities between Kotlin Multiplatform and React Native

The thesis examined the implementation of touch functionalities on two popular development platforms, Kotlin Multiplatform and React Native. Touch functionalities refer to gestures performed on a screen by the user using a finger or another device capable of registering touch.

The objective of the thesis was to determine how the adoption, performance, and functionality of touch interactions differed between the two frameworks. Equivalent application functionalities were developed for both platforms, and observations made during the development formed the basis for a comparative analysis, both by individual aspects and from an overall perspective.

The implementations showed many similarities, and their complex functionalities required manual development due to the limitations of the available gesture detectors. React Native provided more accessible development experience and better online resources, whereas Kotlin Multiplatform offered built-in features that simplified the development work.

Future research could continue by exploring new gesture combinations and by investigating how various third-party libraries might affect the development process and potentially simplify or automate manual tasks.

Keywords:

touch functionalities, React Native, Kotlin, mobile application

Sisältö

Sanasto	7
1 Johdanto	9
2 Opinnäytetyössä käytettävät teknologiat	12
2.1 Kotlin Multiplatform Mobile -kehityskehys	12
2.2 React Native -kehityskehys	13
2.3 Android Studio -kehitysalusta	15
2.4 Expo -kehitysalusta	17
3 Aiempia tutkimuksia ja niiden keskeiset metodit	19
4 Teoriapohjainen vertailu Kotlin Multiplatform Mobilen ja React Nativen välillä	21
4.1 Käyttötarkoituksen eroavaisuudet	21
4.2 Kehityskokemuksen eroavaisuudet	21
4.3 Työkalujen ja riippuvuuksien eroavaisuudet	23
4.4 Testauksen ja virheenselvityksen eroavaisuudet	25
4.5 Suorituskyvylliset erot	26
5 Vertailun toteuttamisen määrittely	29
6 Kehitysalustojen valmisteleva	31
7 Kosketustoimintojen kehittäminen ja vertailu	34
7.1 Sovelluksen aloitusnäky	34
7.1.1 Aloitusnäky kehitys Kotlin Multiplatformilla	35
7.1.2 Aloitusnäky kehitys React Nativella	37
7.1.3 Aloitusnäky kehityksen vertailu	39
7.2 Piirtotoiminnallisuus	42
7.2.1 Piirtotoiminnallisuus Kotlin Multiplatformilla	43
7.2.2 Piirtotoiminnallisuus React Nativella	46
7.2.3 Piirtotoiminnallisuuden vertailu	49

7.3 Palapelitoiminnallisuus	51
7.3.1 Palojen yhtäaikainen siirtäminen ja rotaatio Kotlin Multiplatformilla	53
7.3.2 Palojen yhtäaikainen siirtäminen ja rotaatio React Nativella	55
7.3.3 Palapelitoiminnallisuuden vertailu	57
8 Kokonaisuuden vertailu tulosten pohjalta	60
9 Yhteenveto ja pohdinta	62
Lähteet	64

Kuvat

Kuva 1. Android studion (vasemmalla) ja Wizardin (oikealla) luomien projektirakenteiden erot.	32
Kuva 2. Aloitusnäkyvän hahmotus.	35
Kuva 3. Kotlin Multiplatformin aloitusnäkyvän kosketustoiminnallisuuden koodi.	36
Kuva 4. Reanimatedin (vas.) ja Gesture Handlerin (oik.) eleenkäsittelijät.	38
Kuva 5. Reanimatedin (vas.) ja Gesture Handlerin (oik.) siirtomäärittysten eroavaisuudet.	39
Kuva 6. Piirtosivun yleisilme.	42
Kuva 7. Canvaksen käyttö ja piirtopolkujen renderöinti Kotlin Multiplatformilla.	43
Kuva 8. Uuden polun (<i>Path</i>) asettaminen vanhan päivityksen sijasta.	44
Kuva 9. Kotlin Multiplatformin tapahtumien kuuntelijan toiminnallisuuksia.	45
Kuva 10. Canvaksen käyttö ja piirtopolkujen renderöinti React Nativella.	47
Kuva 11. Apufunktiot ja rakenne piirtogesturessa.	48
Kuva 12. Piirtomuuttujien hallinta ja manipulointi apufunktiossa.	48
Kuva 13. Piirtoalustojen syntaksin vertailu Kotlin Multiplatformin (vasemmalla) ja React Nativen (oikealla) välillä.	50
Kuva 14. Palapelisivun yleisilme.	52

Kuva 15. Elementtien renderöinti palapelisivulla (Kotlin Multiplatform).	54
Kuva 16. Palapelinäkymän piirtoelementit (React Native).	56
Kuva 17. Siirtomuutoksen koodi palapelinäkymässä Kotlin Multiplatformin ja React Nativen välillä.	58

Taulukot

Taulukko 1. Kehityskokemuksen eroavaisuudet.	22
Taulukko 2. Työkalujen ja riippuvuuksien eroavaisuudet.	23
Taulukko 3. Testauksen ja virheenselvityksen eroavaisuudet	25
Taulukko 4. Suorituskyvylliset erot.	26
Taulukko 5. Vertailtavat alueet	30
Taulukko 6. Aloitusnäkymän toteutunut kosketuskeskeinen koodirivimäärä.	40
Taulukko 7. Piirtonäkymän toteutunut kosketuskeskeinen koodirivimäärä.	50
Taulukko 8. Palapelinäkymän toteutunut koodirivimäärä.	58

Sanasto

Breakpoint	Ohjelmointityökalu, jolla suoritus pysäytetään tiettyyn kohtaan koodissa ohjelman käyttäytymisen tarkastelemiseksi.
Data-pipeline	Tietovirta-arkkitehtuuri, jossa dataa siirretään vaiheittain useiden prosessien läpi analysointia, muokkausta tai tallennusta varten.
Drag-and-drop	Käyttöliittymätoiminto, jossa käyttäjä siirtää elementtiä kosketuksella vetämällä ja pudottamalla sen toiseen paikkaan.
Fold	Taitettava mobiilipuhelin, jonka kuvasuhde avattuna on neliömäinen.
FPS	Kuvataajuus, joka kertoo näytettävien kuvien määrän sekunnissa ruudulla.
GPU	Erillinen prosessori, joka on optimoitu käsittelemään graafisia operaatioita.
Hook	React-kehityksessä käytetty funktio, jolla voi liittää komponenttiin tilanhallintaa ilman luokkakomponentteja.
IDE	Ohjelmistokehitysympäristö, joka yhdistää editorin, kääntäjän ja virnehallintatyökalut yhdeksi sovellukseksi.
Just-in-time (JIT)	Käännös menetelmä, missä ohjelmakoodi käännetään suoritettavaksi vasta ajon aikana suorituskyvyn optimoimiseksi.
Logi	Tapahtumakirjaus, mihin ohjelma tallentaa suorituksen aikaisia tietoja.
Null	Arvo, joka merkitsee olematonta arvoa. Yleinen virhelähde ohjelmoinnissa.

Over-the-Air	Teknologia, jolla päivitykset lähetetään langattomasti käyttäjän laitteeseen.
Päästä-päähän testaus	Sovelluksen toiminnallisuuden testaus kokonaisvaltaisesti alusta loppuun.
Renderöityä	Käyttöliittymäkomponentin piirtyminen näytölle.
Snap	Toiminto, jossa liikkuva elementti lukkiutuu automaattisesti tiettyyn kohtaan ruudulla.

1 Johdanto

Kosketustoiminnot ovat keskeinen osa nykyaikaisten mobiilisovellusten käyttöliittymiä. Ne tarjoavat käyttäjälle suoran ja intuitiivisen tavan olla vuorovaikutuksessa digitaalisen sisällön kanssa. Käytännössä kosketustoiminnot ovat responsiivisia komentoja, joiden avulla käyttöjärjestelmä toteuttaa ennalta määriteltyjä toimintoja, kuten visuaalisten elementtien siirtämisen, aktivoimisen tai näkymän kohdistamisen toiseen sijaintiin näytöllä.

Useimmiten kosketustoimintoja ohjataan sormilla, mutta ne voivat aktivoitua myös muilla lämpöä tai painetta tuottavilla pinnoilla. Lisäksi niitä varten on kehitetty erityisiä välineitä, kuten tarkkuutta vaativaan käyttöön soveltuvia stylus-kyniä. (Miller 2020.)

Kosketustoimintojen laajamittainen käyttö näkyy monilla sovellusalueilla. Tunnetuimpia käyttökohteita ovat esimerkiksi valokuvien selaaminen, karttasovellusten zoomaus ja siirto, pelien ohjaus, sekä käyttöliittymän navigointi eri näkymien välillä. Viime vuosina monikosketusteknologian kehittyminen on mahdollistanut entistä monipuolisempien ja rikkaampien käyttöliittymien suunnittelun, joissa käyttäjän useampi sormi voi samanaikaisesti ohjata eri toimintoja (Crudu 2024).

Tässä opinnäytetyössä vertaillaan kosketustoimintojen käyttöönottoa Kotlinin ja React Nativen välillä. Opinnäytetyön tavoitteena on laatia kattava vertailu valittujen ohjelmointikielten kosketustoiminnoista, jotta mobiilisovelluksia suunnittelevat kehittäjät voivat valita tarpeisiinsa parhaiten sopivan vaihtoehdon. Toinen tavoitteeni prosessin aikana on syventää asiantuntemustani kosketustoiminnallisuuksien toteutuksessa, jotta pystyn tulevaisuudessa arvioimaan kehitysprojektien aikatauluja realistisemmin sekä suorittamaan toteutuksia teknisesti vahvalla perusosaamisella. Samalla perehdyn uuteen teknologiaan, Kotlin Multiplatform-kehitysympäristöön, sekä sen taustalla olevaan ohjelmointikieleen Kotliniin, jotka ovat minulle ennestään tuntemattomia.

Työssä käsitellään vertailtavien tekniset eroavaisuudet, niiden ominaiset vahvuudet ja heikkoudet, sekä dokumentoidaan kehittäjäkokemukset selkeäksi ja analyttiseksi kokonaisuudeksi. Lisäksi havainnollistusta tuetaan kuvakaappauksilla, jotka esittelevät sekä kielten syntaksia että sovelluksen toiminnallisuuksien visuaalisia ulkoasuja.

Aiheen aiempaa kirjallisuutta tarkasteltaessa voidaan huomata, että vaikka ohjelmointikielten vertailua on käsitelty laajasti useissa lähteissä, saatavilla oleva materiaali keskittyy lähes aina suorituskyykyyn, syntaksiin, soveltuvuuteen tai suosioon. Sen sijaan tutkimukset, joissa vertaillaan erityisesti kosketustoiminnallisuuksia, ovat hyvin harvinaisia, eikä tätä opinnäytetyötä kirjoitettaessa löytynyt yhtään vastaavaa. Kosketustoiminnallisuudet ovat kuitenkin yhä keskeisemmässä roolissa nykyaikaisissa sovelluksissa, ja olisi hyödyllistä, että myös tästä näkökulmasta olisi enemmän vertailumateriaalia saatavilla kehittäjien tueksi.

Opinnäytetyön menetelmä on tapaustutkimus, minkä lisäksi siinä käytetään kvalitatiivisia sekä kvantitatiivisia menetelmiä. Kvalitatiivinen lähestymistapa näkyy kehittäjän omien kokemusten analysoinnissa ja siinä, kuinka hyvin määriteltäjä toiminnallisuuksia onnistutaan tuottamaan. Kvantitatiivinen osuus puolestaan ilmenee koodirivien ja apufunktioiden määrän numeerisessa vertailussa. Lisäksi tarkastellaan mahdollisia ajallisia eroja sovellusten piirtämisen nopeudessa ja kosketukseen reagoinnissa.

Vertailu toteutetaan sekä teoreettisen verkkomateriaalin pohjalta, että käytännön työskentelyn havaintojen mukaisesti. Opinnäytetyössä käytetään kehitysalustoja, sekä ohjelmointikieliä yksin, tai yhdessä kehityskehyksen kanssa. Käytännön työssä kehitetään yksinkertaisia mobiilisovelluksia, jotka sisältävät monipuolisia kosketuselementtejä riittävän havaintomateriaalin takaamiseksi. Kokonaisuudessaan sovellukset sisältävät kolme näkymää poikkeavilla toiminnallisuuksilla, jotka ovat aloitus-, piirto- sekä palapelinäkymät. Mobiilisovellukset toimivat tutkimusalustoina, eikä niitä kehitetä julkaisumielessä.

Käytännön työn lisäksi hyödynnettävät teknologiat esitellään niiden teoreettisen taustan perusteella. Aihealueesta on myös tehty vastaavanlaisia tutkimuksia, ja tässä työssä esitellään niistä muutamia (Sipola 2023; Heinonen 2020; Nyrhinen 2022; Pereira, Couto, Ribeiro, Rua, Cunha, Fernandes & Saraiva 2017; Ali & Qayyum 2021). Aiempien tutkimusten tulosten perusteella on luotu määrittelyt, miten työn tutkittavia elementtejä vertaillaan ja millaisiin ominaisuuksiin niissä kiinnitetään erityishuomiota.

Opinnäytetyössä hyödynnetään tekoälyä, verkkoresursseja ja omakohtaista intuitiota. OpenAI:n (OpenAI 2025a) tekoälyä käytetään ehdottamaan suosittuja kirjastoratkaisuja kosketustoiminnallisuuden optimoimiseksi molemmilla alustoilla, mikä varmistaa, että vertailu keskittyy niiden parhaisiin mahdollisuuksiin. Tekoälyä käytetään myös koodin alustamiseen, mitä muokataan edelleen verkkoresurssien ja intuitiivisten ratkaisujen pohjalta tehtävän tavoitteiden mukaiseksi. Kirjastojen soveltuvuus varmistetaan sekä tekoälyn ehdottamien lähteiden avulla että henkilökohtaisella tiedonhaulla oikeellisuuden takaamiseksi.

Tulosten perusteella luodaan lopuksi kokonaisvertailu teknologioiden välillä ja annetaan suosituksia siitä, millaisiin käyttötarkoituksiin ja tilanteisiin kukin teknologia sopii parhaiten opinnäytetyön havaintojen perusteella.

2 Opinnäytetyössä käytettävät teknologiat

Ohjelmistokehityksessä käytetään kehityskehyksiä työkaluina tarjoamaan valmiita rakenteita, sekä toiminnallisuuksia sovellusten rakentamiseen. Niiden tarkoitus on nopeuttaa kehitysprosessia tarjoamalla uudelleenkäytettäviä komponentteja sekä ratkaisuja yleisiin ongelmiin. (Awati 2024.)

Kehitysalusta on ohjelmallinen ympäristö, mikä mahdollistaa nopean sovelluksen kehittämisen, testaamisen, sekä käyttöön ottamisen. Se sisältää sekä laitteiston että ohjelmiston, kuten käyttöjärjestelmän, laiteajurit ja muut tarvittavat komponentit. Usein organisaatiot hyödyntävät olemassa olevia kolmannen osapuolen tuottamia kehitysalustoja, mutta myös räätälöity vaihtoehto on mahdollista tuottaa yritystarpeisiin. Tyypillisesti kehitysalusta tarjoaa valmiita käyttöliittymän kehitystyökaluja, sovellusohjelmointirajapintojen hallintatyökaluja, mobiilin taustapalvelimen, sekä tuen emuloiduille laitteille. (Steele & Darling 2023.)

Tässä opinnäytetyössä käytetään kehitysalustoina Android Studiota ja Expoa. Android Studio toimii emulaattorina Expo-pohjaiselle sovellukselle, joka on kirjoitettu React Native -kehityskehyksellä ja JavaScript-ohjelmointikielellä. Kotlin-ohjelmointikielellä toteutettu sovellus käyttää Android Studiota sekä emuloidun laitteen ajonaikaisena ympäristönä että kehitystyön työkaluna.

2.1 Kotlin Multiplatform Mobile -kehityskehys

Kotlin on JetBrainsin vuonna 2011 kehittämä avoimen lähdekoodin ohjelmointikieli, joka ensisijaisesti soveltuu Android-sovellusten kehitykseen, minkä lisäksi sen tarjoama Multiplatform-ominaisuus mahdollistaa koodin siirtämisen eri käyttöjärjestelmien välillä monialustasovellusten kehittämistä varten, kuten esimerkiksi Applen watchOS:ään. (Lutkevich 2024.)

Palvelinpuoli on toinen käyttömahdollisuus Kotlinille koska se on yhteensopiva useiden tunnettujen kehysten kanssa, kuten Springin ja Ktorin (JetBrains 2024).

Kotlinin natiivi koodi ja sen tarjoama suorituskyky johtavat sen suosioon myös datatieteessä, missä Kotlinia käyttäen tuotetaan data-pipeline

ja sekä tekoälymalleja (Lutkevich 2024).

Se on suunniteltu olemaan Javan korvaaja, ja yhtenä kehityselementtinä on sen yksinkertaisempi syntaksi. Koodi muistuttaa hieman muita ohjelmointikieliä ja se on suunniteltu niin, että boilerplate-koodia koostuisi mahdollisimman vähän.

Näiden ansiosta aiempaa ohjelmointitaustaa omaavien kynnys siirtyä työskentelemään Kotlinin kanssa on vähäinen. (Lutkevich 2024.)

Koska Kotlin tukee JavaScriptiä, LLVM:ää ja Just-in-Time-kääntämistä, se mahdollistaa sovellusten siirrettävyyden eri alustoille ja käyttöjärjestelmille kehitystarpeiden mukaisesti. (Lutkevich 2024). JavaScriptiin kääntäminen on eduksi verkkosovellusten kehittämisessä, sen mahdollistaessa sekä backend- että frontend kehityksen samalla standardiohjelmointikielellä (Lutkevich 2024). Kotlin/Native-työkalua voi käyttää yhdessä LLVM:n kanssa kääntämään koodi natiiviksi konekieleksi, mikä mahdollistaa Kotlinin käytön muun muassa iOS-sovelluksissa, joissa suoritettavan koodin tulee olla natiivi muodossa (JetBrains 2024). Kotlin/Nativen käännös tosin tukee vain taustalogiikkaa, jolloin muun muassa UI on toteutettava käyttöjärjestelmäkohtaisella natiivikoodilla. Kotlin on lisäksi täysin yhteensopiva Java-koodin kanssa, minkä takia olemassa olevat Java-projektit ovat asteittain siirrettävissä Kotliniin (Lutkevich 2024).

2.2 React Native -kehityskehys

React Native on Meta Platformsin vuonna 2015 kehittämä avoimen lähdekoodin kehityskehys, jonka avulla voi kehittää mobiilisovelluksia muun muassa iOS:n ja Androidin käyttöliittymille hyödyntäen JavaScriptiä ja Reactin kirjastoa. Vuonna 2018 sillä oli toiseksi eniten myötävaikuttajia GitHubissa, ja tällä hetkellä se on tuettuna useiden suurien yritysten puolesta, kuten Microsoft, Callstack ja Expo. Monia merkittäviä mobiilisovelluksia on tuotettu React Nativella, esimerkiksi

Facebookin, Outlookin, Teamsin, Discordin ja Teslan sovellukset hyödyntävät sitä kehityskehyksenä (React Native 2025.)

React Nativen yksi merkittävä hyöty on sen yhtenäinen koodipohja, mikä tarkoittaa, että sama koodi kääntyy sekä Android että iOS käyttöjärjestelmille (Maciej Budziński 2024). Poikkeuksena tähän on, että alustaspesifit konfiguraatiot on määritettävä erikseen ja tietyt lisäominaisuudet eivät välttämättä ole saataville kummallekin käyttöjärjestelmälle. Myös käyttöliittymän ulkoasu käyttöjärjestelmien välillä voi poiketa toisistaan, erityisesti kolmannen osapuolen kirjastoja käytettäessä. Yhtenäisen alustuskoodin etuna on, että se säästää resursseja ja aikaa mobiilisovelluksia kehitettäessä, sekä mahdollistaa suuremman yleisön saavuttamisen poistamalla tarpeen keskittyä vain tietyn käyttöjärjestelmän omaavaan asiakaskuntaan (Maciej Budziński 2024).

Nyky aikaisten standardien mukaisesti React Native tarjoaa Fast refresh-ominaisuuden, mikä tarkoittaa sitä, että muutokset käyttöliittymän koodissa ovat havaittavissa välittömästi, ilman että sovellusta täytyy uudelleen alustaa. React Nativen koodi on myös nopeaa ja suorituskykyistä (Maciej Budziński 2024). Vaikkei JavaScript olekaan yhtä nopea kuin natiivi koodi, sen ero ihmissilmälle on minimaalista useammissa tapauksissa Swiftiin verrattuna (Chrzanowska 2024).

Käyttöliittymä toteutetaan yhdistelemällä uudelleen käytettäviä komponentteja. Peruslaatuiset yleiskomponentit koostuvat muun muassa Text-komponentista, joka on tarkoitettu tekstin kehikseksi ja View-komponentista mikä on universaali kehys kaiken tyyppiselle sisällölle. Kaikki komponentit sisältävät kattavan kustomoinnin kehittäjän tarpeiden mukaisesti. React Native mahdollistaa natiivien käyttöliittymäelementtien käytön, minkä takia sillä toteutetut sovellukset tarjoavat perinteisin natiivisovellusten kokemuksen. (React Native 2025) Erona muihin alustariippumattomiin vaihtoehtoihin React Nativella on, että WebView'n renderöinnin sijaan, se suoriutuu natiiveilla näkymillä ja komponenteilla. (Maciej Budziński 2024.)

React Nativea käytettäessä yleisiä haasteita ovat sovelluksiin sopivien kustomoitujen moduulien löytämisen vaikeus sekä tietyt toiminnallisuudet, jotka saattavat kokonaan puuttua. Lisäksi julkisesti saatavilla olevista moduuleista voi löytyä virheitä, jotka vaikeuttavat niiden integroimista omiin projekteihin. Tämä voi johtaa ylimääräiseen virheidenkorjaukseen tai jopa pakottaa kehittäjät ylläpitämään erillisiä koodikantoja eri käyttöjärjestelmiä varten. Vaikka React Native skaalautuu useimmissa tapauksissa hyvin sovelluksen kasvaessa monimutkaisemmaksi, joskus natiivikoodiin siirtyminen voi olla välttämätöntä riittävän suorituskyvyn saavuttamiseksi. Esimerkiksi Airbnb luopui React Nativesta, koska se ei vastannut yrityksen kasvun vaatimaa suorituskykyä, ja he päätyivät kehittämään erilliset natiivisovellukset. (Maciej Budziński 2024)

2.3 Android Studio -kehitysalusta

Android Studio on Googlen kehittämä kehitysalusta, jota voidaan terminologisesti kutsua myös kehitysympäristöksi, mikäli sen toimintaan viitataan IDE:nä. Android Studion toiminta pohjautuu JetBrainsin IntelliJ IDEA-alustaan ja sen tavoite on mahdollistaa kehittäjille yhtenäinen työympäristö, joka kattaa suunnitteluun, testaukseen, sekä julkaisuun tarvittavat työkalut Android-sovelluksia kehitettäessä (Ducrohet et al. 2013).

Soveltuvuudeltaan Android Studio on tarkoitettu sovellusten kehittämiseen kaikille Android laitteille hyödyntäen Kotlin- tai Java-ohjelmointikieliä. Se sisältää myös Android Native Development Kitin (NDK), jolla on mahdollista korvata osa sovelluksen logiikasta tai suorituskykykriittisistä komponenteista C++ -ohjelmointikielellä. (Android Developers 2025.)

Koontiprosessi ja riippuvuuksien hallinnointi on automatisoitu Gradle-pohjaiselle rakennusjärjestelmälle, jonka konfiguraatioita on mahdollista kustomoida tarpeiden mukaan. Gradle mahdollistaa useiden APK-tiedostojen luonnin poikkeavilla toiminnoilla käyttäen samaa projektia ja moduuleja, sekä lähdekoodin kierrättämisen eri lähdejoukkojen välillä. (Android Developers 2025.)

Android Studio tarjoamiin työkaluihin sisältyy tehokas koodieritori, jonka ominaisuuksiin kuuluu koodin automaattinen täydennys, virheiden tunnistus sekä korjaaminen, koodianalysoinnin työkalut, dokumentaatio, navigointi, muotoilu, mallikoodit, testaus- sekä ohjelmistovirheiden korjaustyökalut. Visuaalisen rakenteen tehostamiseksi AS sisältää Layout Editorin, jolla on mahdollista lisätä yksinkertaistettuja käyttöliittymäkomponentteja suoraan näkymään drag-and-drop-menetelmällä. Lisäksi Design- ja Split Views-näkymät mahdollistavat asettelun esikatselun reaaliajassa ilman- tai rinnakkain XML-koodin kanssa. (Android Developers 2025.)

Suorituskyvyn optimointiin ja ongelmien selvitykseen Android Studio integroidut virheidenkorjausominaisuudet sisältävät monipuolisia toimintoja, kuten koodin asteittaisen tarkastelun breakpointteja hyödyntäen, sekä muistin- ja tietoliikenteen käytön seuraamisen. Kaikista virheistä sekä varoituksista generoidaan logit, mikä tehostaa virheiden selvitystä. (Android Developers 2025h)

Android Studio tarjoaa myös emuloidun ympäristön mobiileille Android-laitteille. Toiminnallisuudesta löytyy useita eri resoluutioisia vaihtoehtoja niin puhelinten, tablettien, kuin foldien joukosta. Emulaattoria on mahdollista hyödyntää myös kolmannen osapuolen teknologioilla, esimerkiksi Expoilla rakennettuja sovelluksia on mahdollista suorittaa sen kanssa (Expo 2024a).

Teknologiassa on havaittu useita virheitä, joista monet on raportoitu julkisesti Android Studio omille verkkosivuille. Useimmat ongelmat liittyvät Gradle-rakennustyökalun käyttöön ja johtuvat usein versiopäivitysten aiheuttamista muutoksista. Lisäksi suorituskyky voi ajoittain heikentyä, erityisesti Gradlen ladatessa riippuvuuksia verkosta, jolloin heikko verkkoyhteys saattaa vaikuttaa negatiivisesti käyttökokemukseen. (Android Developers 2025c.)

2.4 Expo -kehitysalusta

Expo on kehitysalusta, joka perustuu avoimeen lähdekoodiin ja se on suunniteltu natiivien applikaatioiden kehitykseen Android ja iOS käyttöjärjestelmille JavaScriptillä kirjoitettuna. Expon ekosysteemi sisältää laajan kokoelman komponentteja ja API-kirjastoja, mitkä helpottavat mobiilitoimintojen integroimista, sekä yksinkertaistavat sovelluksen kokoamista ja julkaisemista. (Expo 2025)

Soveltuvuudeltaan Expo on erityisesti ominainen pienille ja keskikokoisille projekteille, joilla ei ole resursseja tai tarvetta natiivikehitykseen.

Expo sisältää tuen reaaliaikaiselle kehitykselle, koodimuutosten reflektoituessa välittömästi käyttöliittymällä. Lisäksi kehitysalusta suorittaa toistuvaa validointia taustalla, ja ilmoittaa virheistä sekä käyttöliittymällä, että terminaalissa. (Development and Production Modes 2024.)

Kehitys Android ja iOS käyttöjärjestelmille on vaivatonta, Expon mahdollistaessa yksinkertainen integrointi muun muassa Android Studion, sekä Xcoden emuloituihin laitteisiin. Emuloitujen laitteiden lisäksi Expo Go-sovellus mahdollistaa sovelluksen kehityskäytön fyysisellä mobiililaitteella ilman natiivien kehitysympäristöjen asennusta (Expo 2025b).

Expo hyödyntää lisäksi Over-the-Air (OTA)-päivityksiä olemassa oleville sovellusversiolle, mitkä nopeuttavat kriittisten virheiden korjaamista, sekä parannusten julkaisemista suoraan käyttäjälle. Tämä tarkoittaa sitä, että muutokset mitkä eivät vaikuta natiivikoodiin, konfiguraatioihin, tai käyttölupiin, voi päivittää ilman sovelluskauppojen tarkasteluprosessia. (Digital 2023.)

Expolla on useita rajoitteita, kuten rajallinen pääsy natiivimoduuleihin, mikä estää tiettyjen toimintojen toteuttamisen ilman monimutkaista eject-prosessia (Reddit n.d.). Lisäksi Expon sisältämät lukuisat valmiit ominaisuudet voivat kasvattaa sovellustiedostojen kokoa verrattuna muihin teknologioihin. Suuret sovellukset voivat myös kohdata haasteita, sillä EAS (Expo Application

Services) -rakennusprosessi keskeytyy tietyn aikarajan ylittyessä, mikä voi monimutkaistaa sovelluksen rakennusta entisestään. (Expo 2025d)

3 Aiempia tutkimuksia ja niiden keskeiset metodit

Ohjelmointikielien ja kehityskehysten vertailusta on tehty useampia tutkimuksia eriävissä asiayhteyksissä, joissa usein esiintyy muutamia yhtenäisiä vertailukohteita. Muun muassa suorituskky on yleinen vertailukohde niitä verrattaessa, sen tutkimukset on esimerkiksi toteutettu Ali Saqibin ja Qayyum Sammarin (2021, 6) työssä vertaillen kielten tehokkuutta, koodirivejä ja suoritusaikaa raskaita laskutoimituksia suoritettaessa. Tutkimuksen tarkkuuden sekä uudelleen toteutettavuuden mahdollistamiseksi käytetyt laitteet kirjataan ulos, ja testit suoritetaan viisi kertaa. (Ali & Qayyum 2021.)

Toisessa tutkimuksessa tutkittiin ohjelmointikielien energiatehokkuutta. Pereira ja kollegat (2017) seurasivat useamman ohjelmointikielen energiankulutusta verraten niiden suorituskkyä, suoritussnopeutta, sekä muistinkäyttöä.

Tutkimuksen keskeisiä kysymyksiä olivat, voiko vertailua tehdä energiatehokkuuden osalta, korreloiko suoritussnopeus energiatehokkuuden kanssa, vaikuttaako muistinkäyttö energiankulutukseen, sekä voidaanko tarkasteltavia elementtejä käyttää kielen paremmuuden indikaattorina.

Tutkimuksessa hyödynnettiin olemassa olevaa testausympäristöä The Computer Language Benchmark Game (CLBG), sekä Intelin Running Average Power Limit-työkalua (RAPL). CLBG mittaa ajonaikaista suorituskkyä suorittamalla tunnettuja ohjelmointiongelmia tarkasteltavalla kielellä, kun taas RAPL-työkalulla voidaan mitata energiankulutusta.

Tutkimuksen virhemarginaalin pienentämiseksi testit ajettiin kymmenen kertaa, jotta välttyttiin mm. kylmäkäynnistyksen tai cache-efektin aiheuttamilta tulospääristymiltä. (Pereira et al. 2017.)

Heinonen sekä Nyrhinen (2020; 2022) puolestaan vertailivat ohjelmointikielten syntaksien välisiä eroja. Syntaksi tarkkailuun on tutkimuksissa valittu kirjoittajan valitsemissa elementtejä, usein alkeiskonsepteja, sillä vertailtavia kohteita kielten syntaksissa on erittäin laajasti. (Heinonen 2020; Nyrhinen 2022.)

Heinonen (2021, 7) vertaili tutkimuksessaan lisäksi kielten käyttökohteita, suosiota, sekä kirjastojen tarjontaa ja implementointia. Käyttökohteita

tarkasteltaessa keskityttiin verkkosovelluksiin ja koneoppimiseen.

Web-sovelluksissa painotettiin joustavuutta käyttäjän näkökulmasta sekä toiminnallisuuksien toteutettavuutta kummallakin teknologialla. Koneoppimisen osalta ohjelmointikieliä vertailtiin niiden yleisyyden perusteella alalla, ja kirjastoja arvioitiin sekä niiden määrän että käytettävyyden näkökulmasta. Suosion vertailu tehtiin hyödyntämällä GitHubin Octoverse-palvelua, joka tarjoaa tietoa ohjelmointikielten käytöstä GitHubin käyttäjien keskuudessa. Lisäksi tutkittiin työpaikkojen määrää kunkin kielen avainsanoilla Stack Overflow'n työnhakukoneessa. (Heinonen 2020, 7–11.)

Sipola (2023) nostaa tutkimuksessaan esille kehittäjäkokemuksen tärkeyden ja selvittää miten sitä voidaan verrata ohjelmointikielten välillä. Kehittäjäkokemus ohjelmistokehittäjälle kuvastaa tämän havaintoja ja tuntemuksia teknistä työkalua käytettäessä. Sen tavoite on saada käytetty teknologia yleistymään alalla tuottamalla käyttäjälle sellaisen tunteen, että työkalun pariin on miellyttävä palata uudestaan ja sitä voisi suositella toisille. (Sipola 2023, 10–11)

Kehittäjäkokemuksen vertailussa arvioidaan teknologian toimivuutta, vakautta, käytön helppoutta ja selkeyttä. Toimivuudella tarkoitetaan teknologian kykyä suoriutua monipuolisesti sen aihepiiriin liittyvistä tehtävistä tai täyttää yleiset toiminnalliset odotukset. Vakaa teknologia puolestaan viittaa suorituskyykyyn ja luotettavuuteen, jossa käyttäjä voi luottaa järjestelmän saumattomaan toimintaan. Ohjelmistovirheet tulisi korjata nopeasti, ja niihin liittyvän tiedon tulisi olla käyttäjille läpinäkyvää. (Cavalcante 2019; Sipola 2023, 10–11.)

Käytön helppous tarkoittaa kehitysprosessin sujuvuutta, jossa käyttäjän tulisi pystyä navigoimaan järjestelmässä vaivattomasti ja hyödyntämään tarvittavia resursseja ja ominaisuuksia kussakin kehitysvaiheessa. Tällaisia voivat olla esimerkiksi verkkodokumentaatio, lokitiedostojen hallinta tai käyttöliittymän apuvälineet. Selkeyden kannalta teknologian tulisi tehdä syy-seuraussuhteet sen käytön aikana selkeiksi ja pitää käyttöliittymä mahdollisimman intuitiivisena ja helposti ymmärrettävänä. (Cavalcante 2019; Sipola 2023, 10–11.)

4 Teoriapohjainen vertailu Kotlin Multiplatform Mobilen ja React Nativen välillä

Kotlin Multiplatform Mobile ja React Native eroavat merkittävästi jo perusteissaan. Kotlin Multiplatform käyttää ohjelmointikielenä käyttöjärjestelmäkohtaisia natiivikieliä sekä Kotlinia, kun taas React Native perustuu Reactiin ja JavaScriptiin (Polus 2024). Näiden työkalujen käyttötarkoitukset eroavat toisistaan, vaikka niiden väliset rajat ovat viime aikoina vähentyneet.

4.1 Käyttötarkoituksen eroavaisuudet

Kotlin Multiplatform ei täysin poista tarvetta käyttää muita natiivikieliä alustakohtaisissa osissa, kuten käyttöliittymän toteutuksessa. React Native puolestaan mahdollistaa sovelluksen kehittämisen eri käyttöjärjestelmille (kuten iOS ja Android) käyttäen pelkästään JavaScriptiä ja Reactia, jolloin keskimäärin 80–90 % koodikannasta voidaan jakaa ilman natiivikielten tarvetta. (Katariya 2023; Polus 2024)

Mobiilisovellusten lisäksi Kotlin Multiplatformia voidaan käyttää palvelinpuolen kehitykseen ja tietokonesovelluksiin, mikä ei ole mahdollista React Nativella. Vaikka niiden yhteisön sekä tunnettavuuden suhteen onkin eroja, kumpaakin käytetään tunnetuissa sovelluksissa. Esimerkiksi Netflixin Android-pohjainen käyttö perustuu Kotliniin ja Instagramin mobiilit implementaatiot ovat React Nativea. (Felixille 2025)

4.2 Kehityskokemuksen eroavaisuudet

Kappaleessa 4.2 käsitelty erot on tiivistetty taulukkoon 1.

Taulukko 1. Kehityskokemuksen eroavaisuudet.

Ominaisuus	React Native	Kotlin Multiplatform
Hot Reloading	Koodimuutokset näkyvät heti.	Ei vastaavaa ominaisuutta.
Ylläpidettävyys	Nopea kehittyminen voi aiheuttaa ongelmia päivittäessä.	Vähemmän teknistä velkaa pitkällä aikavälillä.
Oppimiskynnys	Helppo oppia JavaScript- tai React-taustaisille.	Helppo oppia Java- tai Swift-taustaisille. Monimutkaistuu monialustakehityksessä.

Kehityskokemukseltaan React Native tarjoaa saumattomamman kokemuksen Hot Reloading-ominaisuuden takia, mikä reflektoi koodimuutokset välittömästi muuttamatta koko applikaation tilaa. Tämä on erityisen tehokas käyttöliittymää toteutettaessa, sen poistaessa tarpeen sovelluksen uudelleen käynnistämiseksi, sekä mahdolliselle navigoinnille työstettävään näkymään. Vaikka Kotlin Multiplatform ei tarjoa React Nativeen verrattavaa ominaisuutta, sen suunnittelu tukee kehittäjän ajankäyttöä painottamalla pitkäaikaista koodin ylläpitoa. Kotlin Multiplatformin selkeä ja moderni syntaksi, null-turvallisuus ja natiivialustojen tuki vähentävät teknisen velan kertymistä ja pienentävät riskiä, että valmis koodi vaatisi merkittäviä muutoksia ajan kuluessa. Toisaalta React Native nojaa nopeasti kehittyvään JavaScript- ja React-ekosysteemiin, mikä voi johtaa tilanteisiin, joissa versiopäivitykset tai alustakohtaiset kirjastot vaativat kehittäjältä huomattavia toimenpiteitä sovelluksen ylläpitämiseksi ja yhteensopivuuden säilyttämiseksi. (Polus 2024.)

Kummankin käyttö on helppo oppia, jos kehittäjällä on aiempaa taustaa tietyissä ohjelmointikielissä. React Native on erityisen nopea omaksua niille, joilla on kokemusta Reactista tai JavaScriptistä, mutta ilman näitä taustatietoja sen opettelu voi vaatia enemmän aikaa. Kotlin muistuttaa monin tavoin sekä Javaa

että Swiftiä, ja jos kehitys keskittyy pelkästään Androidiin, Kotlinin hallitseminen riittää. Sen sijaan, jos tavoitteena on kehittää useille käyttöjärjestelmille Kotlin Multiplatformilla, projektin monimutkaisuus kasvaa, koska mukana on enemmän natiivikieliä. (Polus 2024.)

4.3 Työkalujen ja riippuvuuksien eroavaisuudet

Kappaleessa 4.3 käsitelty erot on tiivistetty taulukkoon 2.

Taulukko 2. Työkalujen ja riippuvuuksien eroavaisuudet.

Ominaisuus	React Native	Kotlin Multiplatform
IDE:t	Monipuoliset vaihtoehdot	Vahvasti sidottu Android-ekosysteemiin
Riippuvuuksien hallinta	npm, Yarn. Laaja kirjastovalikoima.	Gradle. Vahva integraatio Android-ympäristöön.
Tyypitys	TypeScript	Kotlinin staattinen tyypitys käännösaikana
Koodin tarkastus	ESLint ja Prettier suosittuja	Sisäänrakennettu Android Studiolla ja IntelliJ IDElla.
Työkalujen joustavuus	Joustava työkalujen valinnassa.	Vahvasti sidottu Android-ekosysteemiin.

Käytettävät IDE:t sekä Kotlin Multiplatformin että React Nativen välillä eroavat toisistaan, ja React Nativen kehityksessä on enemmän joustavuutta työkalujen valinnassa, kun taas Kotlin Multiplatform-kehitys on vahvemmin sidoksissa Android-ekosysteemiin. Kotlin-kehitystä varten suosituimmat vaihtoehdot ovat Android Studio ja IntelliJ IDEA, jotka tarjoavat natiivin tuen Kotlin-koodille sekä kattavat työkalut Android-sovelluskehitykseen. React Native-kehityksessä IDE-vaihtoehtoja on enemmän, ja suosituimpia ovat VS Code, WebStorm ja

myös Android Studio, jotka kaikki tarjoavat laajan valikoiman työkaluja JavaScript- ja React-koodin hallintaan. (Polus 2024.)

Riippuvuuksien hallinta eroaa myös näiden kahden välillä. Kotlin Multiplatform käyttää Gradle-rakennustyökalua, joka on vahvasti integroitu Android-ekosysteemiin ja tarjoaa monipuoliset mahdollisuudet riippuvuuksien määrittämiseen ja projektin hallintaan. React Native puolestaan hyödyntää yleisemmin JavaScript-ekosysteemin työkaluja, kuten Node Package Manageria ja Yarnia, jotka mahdollistavat riippuvuuksien nopean asentamisen ja hallinnan. React Nativelle on saatavilla paljon kustomoituja kirjastoja sekä lisäosia eri käyttötarkoituksiin, kun taas Kotlin Multiplatformilla ratkaisujen löytyminen nojaa vahvemmin Kotlinin omaan ekosysteemiin, mikä on erityisesti Androidin puolella merkittävä. (Polus 2024.)

React Native ja Kotlin Multiplatform eroavat merkittävästi tyyppityksen toteutuksessa ja linttausominaisuuksissa. React Nativessa tyyppitys toteutetaan usein TypeScriptin avulla, joka on JavaScriptin päälle rakennettu tyyplitetty kieli. TypeScript parantaa koodin ennakoitavuutta ja vähentää virheitä, mutta koska se ei ole natiivisti osa JavaScriptiä, kehittäjiä on lisättävä TypeScriptin käyttö projektiin erikseen ja varmistettava, että koodi kääntyy virheettömästi JavaScriptiksi (React Native 2025).

Kotlin puolestaan sisältää vahvan, staattisen tyyppitysjärjestelmän, joka on syvästi integroitu kieleen. Tämä tarkoittaa, että kaikki tyypit tarkistetaan käännösaikana, mikä vähentää runtime-virheiden mahdollisuutta. Lisäksi Kotlinin Null Safety -ominaisuus suojaa tehokkaasti null-viittausvirheiltä, jotka ovat yksi yleisimmistä virhelähteistä ohjelmoinnissa. Null Safety varmistaa, että muuttujat määritellään joko ei-null-tyyppeinä tai eksplisiittisesti null-arvoja sallivina tyyppeinä, mikä pakottaa kehittäjän käsittelemään mahdolliset null-arvot etukäteen. (Akhin & Belyaev 2025)

Koodin tarkastuksessa React Native hyödyntää JavaScript-ja TypeScript-maailmassa suosittuja työkaluja, kuten ESLint ja Prettier, jotka auttavat ylläpitämään koodin laatua ja yhdenmukaisuutta automaattisesti (Kasle

2024). Kotlin puolestaan tarjoaa sisäänrakennetun koodin tarkastus ratkaisun Android Studioon ja IntelliJ IDEAan, joissa on myös kehittyneet tyyliohjeiden tarkistukset ja varoitukset (Android Developers 2025d). Lisäksi Kotlinissa voidaan käyttää ulkoisia työkaluja, kuten ktlint, ylläpitämään koodin tyyliä ja linjaamaan se parhaita käytäntöjä vastaavaksi (Ktlint n.d.).

4.4 Testauksen ja virheenselvityksen eroavaisuudet

Kappaleessa 4.4 käsitellyt erot on tiivistetty taulukkoon 3.

Taulukko 3. Testauksen ja virheenselvityksen eroavaisuudet

Ominaisuus	React Native	Kotlin Multiplatform
Yksikkötestaus	Jest	Ei sisäänrakennettu, käytetään alustan omia kehyksiä
Päästä päähän -testaus	Detox	Ei suoraa tukea, käytetään alustan työkaluja
Komponenttitestaus	React Testing Library	Riippuu käyttöliittymän alustasta
Virheenselvitystyökalut	React Developer Tools	Android Studio, IntelliJ IDEA, Xcode
Konfiguroitavuus	Vähemmän kehittäjän konfiguroitavaa	Enemmän kehittäjän vastuulla

React Native ja Kotlin Multiplatform eroavat merkittävästi testaustyökaluissaan ja menetelmissään. React Native tukee useita testauskehyksiä eri tarkoituksiin, kuten Jest yksikkötestaukseen, Detox päästä päähän-testaamiseen ja React Testing Library komponenttien käyttäytymisen testaamiseen. Lisäksi React

Native sisältää sisäänrakennettuja virheenselvitystyökaluja, kuten React Developer Tools, jotka helpottavat sovelluksen tilan ja komponenttien tarkastelua reaaliaikaisesti. (Polus 2024)

Kotlin Multiplatform sen sijaan ei tarjoa sisäänrakennettuja testauskehyksiä, vaan kehittäjät käyttävät yleensä käyttöliittymäkohtaisia testauskirjastoja, kuten Espresso Android-sovellusten testaamiseen. Virheidenkorjausta varten Kotlin-kehittäjät hyödyntävät Android Studion ja IntelliJ IDEAn tarjoamia kattavia työkaluja, jotka tukevat esimerkiksi virheiden jäljittämistä ja suorituskyvyn analysointia. iOS-sovelluksia varten Kotlin Multiplatform-kehittäjät voivat hyödyntää Xcoden virheidenkorjaustyökaluja. (Polus 2024)

React Native tarjoaa laajemman valikoiman kehyksiä ja työkaluja dynaamisiin ja monialustaisiin sovelluksiin, kun taas Kotlin Multiplatformin testausratkaisut keskittyvät yleensä alustan erityistarpeisiin, mikä tekee siitä joustavan mutta enemmän kehittäjän konfigurointia vaativan vaihtoehdon. (Polus 2024)

4.5 Suorituskyvylliset erot

Kappaleessa 4.5 käsitellyt erot on tiivistetty taulukkoon 4.

Taulukko 4. Suorituskyvylliset erot.

Ominaisuus	React Native	Kotlin Multiplatform
Suorituskyky	Hyvä käyttöliittymä painotteisissa sovelluksissa. Hyödyntää natiiveja UI-komponentteja.	Hyvä komputaatio painotteisissa sovelluksissa natiivin API:en ja muistinhallinnan ansiosta.
Käännöstapa	JavaScript Just-in-Time käännös parantaa suorituskykyä dynaamisesti.	Käännetään suoraan natiiviksi. Vakaa ja tehokas.
UI-optimointi	Vain tarpeelliset komponentit päivitetään.	Ei automaattista optimointia, riippuu käytetyistä työkaluista.

Muistinhallinta	Riippuvainen JavaScriptin muistinhallinnasta.	Integroitu. Vähentää muistivuotoja ja fragmentaatiota.
-----------------	---	--

Kotlin Multiplatform erottuu edukseen suorituskyvyltään erityisesti komputaatiollisesti raskaissa tehtävissä, koska se tarjoaa suoran pääsyn natiiveihin API-rajapintoihin ja funktioihin. Tämä mahdollistaa korkean suorituskyvyn niin graafisesti vaativissa sovelluksissa kuin reaaliaikaisissa interaktioissa. Lisäksi Kotlinin tehokas muistinhallinta vähentää muistivuotojen ja fragmentaatio-ongelmien riskiä pitkään käynnissä olevissa sovelluksissa. Tämä varmistaa tasaisen suorituskyvyn erityisesti laitteissa, joissa muistin määrä on rajoitettu. (Polus 2024.)

React Native puolestaan loistaa käyttöliittymiltään raskaissa sovelluksissa, sillä se hyödyntää alustakohtaisia natiiveja UI-komponentteja tehokkaasti. Lisäksi sen suorituskyykyä parantaa JavaScript-koodin suorittamiseen käytettävä Just-In-Time (JIT)-kääntäminen (Polus 2024). Kun sovelluksessa tapahtuu muutoksia, React Native rakentaa hierarkian käyttöliittymäkomponenteista ja optimoi päivitykset vertaamalla aiempaa ja nykyistä tilaa. Tämä prosessi muistuttaa React.js:n virtual DOM-mallia, mutta on mukautettu toimimaan natiivien käyttöliittymäkomponenttien kanssa (StackOverflow 2019). Näin vain välttämättömät komponentit päivitetään, mikä säästää aikaa ja vähentää suorituskyykyyn kohdistuvaa raskautta, koska koko käyttöliittymää ei tarvitse renderöidä uudelleen (Polus 2024).

Toisin kuin React Native, Kotlin ei sisällä automaattisesti vastaavaa optimointimekanismia. Kotlinin toiminta perustuu suoraan natiivien API-rajapintojen ja komponenttien hyödyntämiseen, mikä voi johtaa parempaan suorituskyykyyn staattisissa ja yksinkertaisemmissa käyttöliittymissä (Polus 2024). Kuitenkin ilman erityisiä optimointimenetelmiä, kuten Jetpack Compose-kirjaston käyttöä, Kotlinin käyttöliittymän päivitykset voivat olla tehottomampia verrattuna React Nativessa käytettyihin automaattisiin

optimointeihin dynaamisissa ja monimutkaisissa sovelluksissa (Android Developers 2025).

5 Vertailun toteuttamisen määrittely

Vertailu perustuu osittain aiemmin mainittuihin lähteisiin, ja menetelmät on mukautettu opinnäytetyön aihepiiriin, eli kosketustoiminnallisuuden tarkasteluun. Koska vertailtavien teknologioiden syntaksi eroaa merkittävästi, analysoidaan koodin ymmärrettävyyttä rivien pituuden sekä syntyneiden apufunktioiden määrän ja selkeyden perusteella. Apufunktioiden määrän kasvaessa koodin kompleksisuus kasvaa, minkä vuoksi se on valittu erilliseksi tarkastelun kohteeksi.

Seuraavaksi kehittäjäkokemusta arvioidaan hyödyntäen Cavalcanten (2019) määritteitä. Toiminnallisuuden osalta tarkastellaan, pystyvätkö teknologiat toteuttamaan halutut ominaisuudet ja kuinka ne suoriutuvat yleisesti kosketustoiminnoista verrattuna odotuksiin. Yleisesti odotetaan, että vastaavan suosion omaavat teknologiat mahdollistaisivat kahden kosketuselementin samanaikaisen liikuttamisen, ilman merkittäviä toteuttamisvaikeuksia. Vakauden arviointi keskittyy siihen, kuinka tasaisesti elementtien siirtäminen onnistuu ja kuinka johdonmukaisesti kosketukset tunnistetaan. Lisäksi havainnoidaan mahdollinen viive joko kosketukseen reagoimisessa tai elementtien piirtämisessä (Taulukko 1).

Kosketustoiminnallisuuksien käyttöönoton helppoutta mitataan arvioimalla, kuinka vaivattomasti verkkoresurssit ovat hyödynnettävissä sovelluskehityksessä ja syntykö prosessin aikana tarpeita merkittäville muutoksille löydettyssä koodissa. Selkeyttä tarkastellaan seuraamalla, kuinka intuitiivista on tehdä muutoksia tai lisätä elementtejä ilman ulkoisten resurssien käyttöä. Sen lisäksi että vertailtavia rinnastetaan toisiinsa, myös niiden yksilökohtaiset negatiiviset sekä positiiviset huomiot prosessin aikana tuodaan esille (Taulukko 1).

Taulukko 5. Vertailtavat alueet

Vertailualue	Kuvaus
Toteutuksen eteneminen	Miten kehitystyö eteni ja millaisia haasteita ilmeni toteutuksen aikana.
Koodirakenne ja apufunktiot	Syntaksin selkeys, apufunktioiden määrä sekä koodin rakenteellinen ero.
Kehittäjäkokemus	Kokonaisvaikutelma kehittäjän näkökulmasta, mm. työskentelyn sujuvuus.
Dokumentaatio ja saatavuus	Dokumentaation laatu, hyödyllisyys ja sen saatavuus kehityksen tukena.
Suorituskyky	Sovelluksen toiminnan nopeus ja responsiivisuus.
Muutosten intuitiivisuus	Kuinka helposti ja luonnollisesti muutoksia pystyi tekemään.
Toteutuksen onnistuminen	Kuinka hyvin lopputulos vastasi tavoitteita ja toiminnallisia vaatimuksia.

Vertailu edistyy siten, että ensin yksi kolmesta tehtäväkategoriasta toteutetaan aloittaen Kotlin Multiplatformilla ja sitten React Nativella, minkä aikana analysoidaan kyseisen vertailtavan suoriutumista prosessissa, sekä sen kehityksen aikaista kompleksisuutta. Kun tämä on kummankin kohdalla toteutettu, verrataan niitä keskenään kaikilla määrittelyn mukaisilla osa-alueilla. Tehtäväkategoriat soveltavassa osuudessa ovat sovelluksen aloitusnäkyvä, piirtotoiminnallisuus, sekä palapelitoiminnallisuus, mitkä toteutetaan mainitussa järjestyksessä.

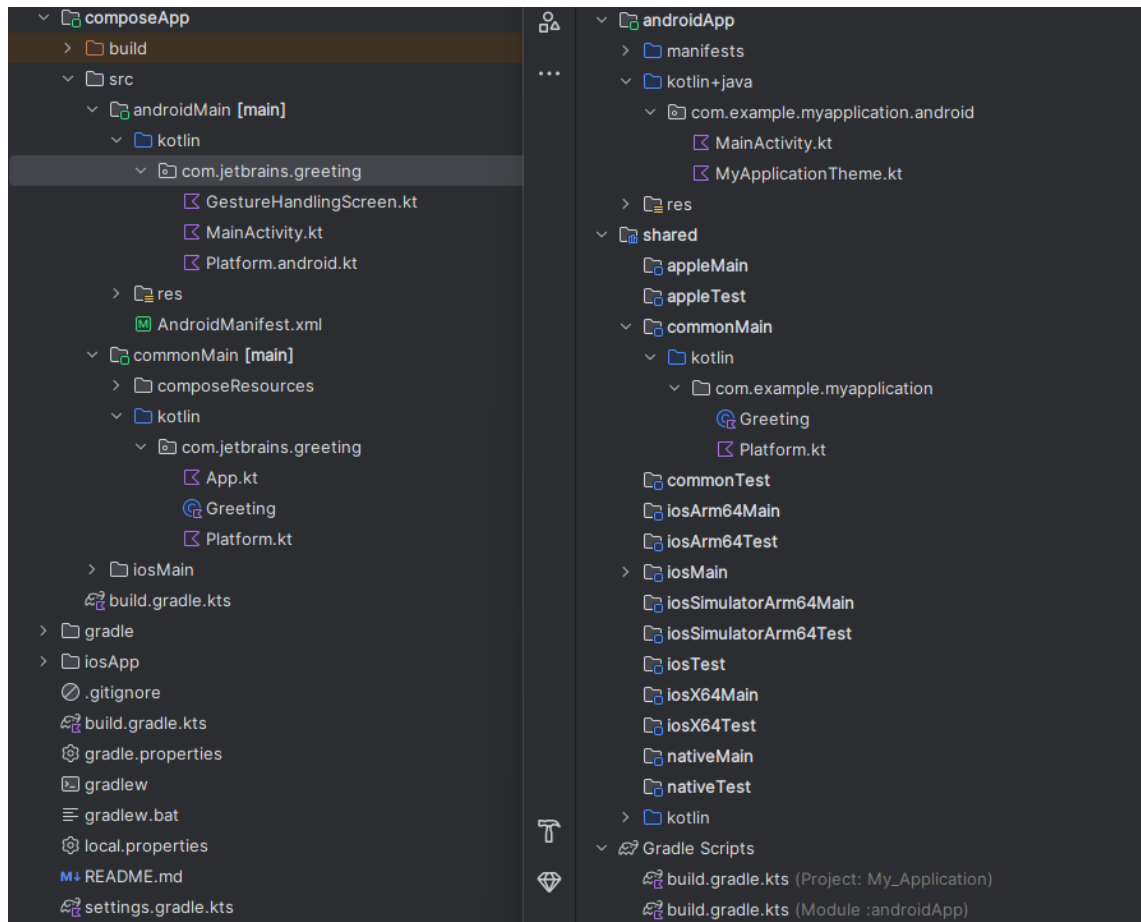
6 Kehitysalustojen valmisteleminen

Kotlin Multiplatform -kehitystä varten käytetään Android Studiota (Android Developers 2025h), jonka voi ladata helposti virallisilta verkkosivuilta. Asennus sujui ongelmitta, mutta itse projektin alustaminen osoittautui yllättävän haastavaksi. Ensimmäinen intuitiivinen ratkaisu oli käyttää Android Studion tarjoamaa Kotlin Multiplatform App-templaattia, jonka avulla projekti saatiin nopeasti toimintakuntoiseksi käyttöliittymän visualisointiin.

Tässä vaiheessa ilmeni kuitenkin ongelma, sillä suurin osa verkosta löytyvästä materiaalista ei ollut suoraan yhteensopiva templaatin luoman rakenteen kanssa, mikä teki projektin muokkaamisesta hankalaa ilman aiempaa kokemusta. Verkko-oppaat ja tutoriaalit näyttivät sen sijaan ohjaavan lähes yksinomaan käyttämään Kotlin Multiplatform Wizardia (JetBrains 2025), joka luo projektin vaivattomasti ja mahdollistaa sen helpon liittämisen Android Studioon.

Wizardin avulla luodun projektin kanssa ilmeni kuitenkin heti alkuun haaste, kun projekti ei synkronoitunut oikein, mikä esti sen käynnistämisen. Ongelman syyksi paljastui puutteellinen SDK-rakenne, joka saatiin korjattua asentamalla uudelleen Android SDK:n oikea versio. Tämän jälkeen halutun työympäristön ohjelmointi onnistui helposti tekoälyn avustuksella.

Verkosta ei löytynyt selkeää materiaalia, joka käsittelisi Kotlin Multiplatform Wizardin ja Android Studion sisäänrakennetun templaatin eroja (Kuva 1). Kysyttäessä tekoälyltä ilmeni, että sisäänrakennettu template vaatii enemmän manuaalista konfigurointia, kun taas Wizard hoitaa suurimman osan asetuksista automaattisesti, mikä tekee siitä käyttäjäystävällisemmän vaihtoehdon erityisesti aloittelijoille (OpenAI 2025b).



Kuva 1. Android studion (vasemmalla) ja Wizardin (oikealla) luomien projektirakenteiden erot.

Expo-kehitysalusta asennettiin käyttämällä NPM:ää (Node Package Manager) komentorivin kautta. Asennuksen jälkeen työympäristö alustettiin tekoälyn avulla, ja siihen sisällytettiin Gesture Handler-kirjasto (Gesture Handler 2025), joka tarjoaa kehittyneempiä toiminnallisuuksia kuin React Nativen sisäänrakennetut ratkaisut (Software Mansion n.d.). Visuaalinen esitys toteutettiin Android Studio emulaattorilla, johon käynnistetty Expo-prosessi yhdistyy automaattisesti. Expo-projektin käyttöönotto sujui ilman ongelmia, mikä saattoi osittain johtua aiemmasta kokemuksesta sen parissa.

Kosketustoiminnallisuuden osalta alustava taustarakenne oli yhtä yksinkertainen toteuttaa kummallakin kehitysalustalla, ja niiden syntaksirivien määrä oli hyvin samanlainen. Kuitenkin kosketustoiminnallisuuksien optimointiin

soveltuvien kirjastojen löydettävyyys poikkesi toisistaan. React Native-puolella useampi ensimmäisillä hauilla löytynyt lähde mainitsi Gesture Handler-kirjaston, kun taas Kotlin Multiplatformin osalta Jetpack Composeen päätyminen ei ollut yhtä suoraviivaista. Etsimällä parasta ratkaisua suoraan Kotlin Multiplatformiin ei johtanut suoraan Jetpack Composen löytämiseen, mutta laajemmalla haulilla Kotlinille ja Androidille löytyi ratkaisu Android Developersin kautta. Ohjeistusta sekä Gesture Handlerin, että Jetpack Composen käyttämiseen löytyi kummassakin tapauksessa runsaasti ja monipuolisesti.

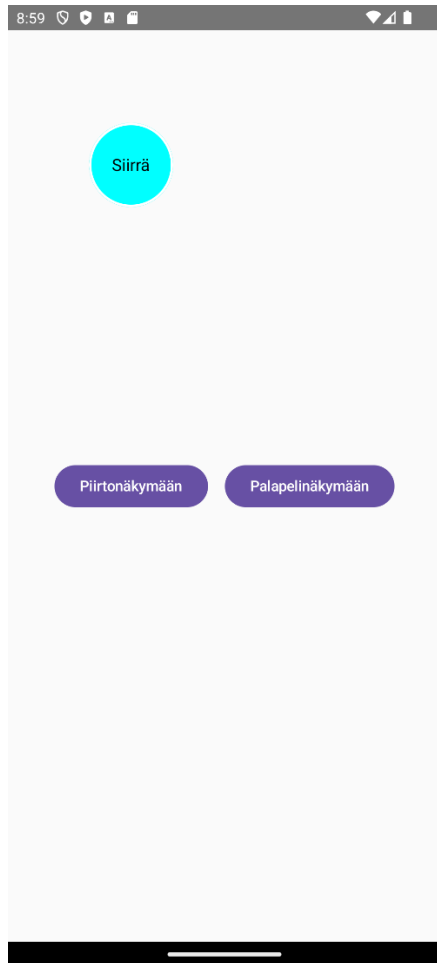
7 Kosketustoimintojen kehittäminen ja vertailu

Soveltavassa osuudessa tarkastellaan kosketustoiminnallisuuksia React Nativen ja Kotlin Multiplatformin välillä, sekä verrataan näitä toimintoja käytännön kehittämistilanteiden valossa. Vertailu keskittyy niihin elementteihin ja periaatteisiin, jotka on määritelty luvun 5 perusteella, ja tutkimuksessa hyödynnetään luvun 2 teoreettista taustaa. Erityisesti verrataan näiden kahden teknologian eroja ja yhtäläisyyksiä kosketustapahtumien hallinnan, selkeyden ja toteutettavuuden näkökulmista. Vertailulähteenä käytetään toteutuksen aikaisia kokemuksia ja haasteita, sekä verkon kautta löydettyä materiaalia.

Ensimmäisessä vaiheessa tarkastellaan aloitusnäkymän toteutusta, jonka jälkeen siirrytään piirtonäkymän kehittämiseen. Viimeisessä osassa käsitellään palapelinäkymän toteutusta.

7.1 Sovelluksen aloitusnäkö

Aloitusnäkö sisältää kaksi violettiä nappulaa, jotka navigoivat käyttäjän joko piirto- tai palapeli näkymiin (Kuva 2). Napit aktivoidaan vetämällä erillistä turkoosia siirtoelementtiä koskettamaan niitä, ja reaktion tulee huomioida osuma sekä nopeassa että hitaassa liikkeessä. Lisäksi siirtoelementtiä kustomoidaan niin, että sen koko suurenee väliaikaisesti, kun vetäminen on aktiivista.



Kuva 2. Aloitusnäköm hahmotus.

Kehitys ja visuaalinen havainnointi toteutetaan Android Studion emuloidulla puhelimella, sillä kaikki tarvittavat toiminnot ovat sillä saavutettavissa.

7.1.1 Aloitusnäköm kehitys Kotlin Multiplatformilla

Kotlin Multiplatformilla aloitusnäköm alustus sujui vaivattomasti tekoälyn avulla, ja ainoa tarkennusta vaatinut kohta liittyi osuman tarkempaan havaitsemiseen. Toteutuksessa sekä napit että siirtoelementti on määritelty kehyksiksi, ja kun niiden rajat osuvat toisiinsa, navigointi tapahtuu (Kuva 3).

```

.pointerInput(Unit) {
    detectDragGestures(
        onDragStart = {
            isDragging = true // Seuraa elementin koon muutosta
            haptic.performHapticFeedback(HapticFeedbackType.LongPress)
        },
        onDrag = { change, dragAmount ->
            change.consume()
            offset.value += dragAmount

            val draggableBounds =
                Rect(
                    offset.value.x,
                    offset.value.y,
                    right: offset.value.x + 100.dp.toPx(),
                    bottom: offset.value.y + 100.dp.toPx()
                )

            button1Bounds.value?.let {
                if (draggableBounds.overlaps(it)) {
                    navController.navigate(route: "piirto")
                }
            }
        }
    )
}

```

Kuva 3. Kotlin Multiplatformin aloitusnäytön kosketustoiminnallisuuden koodi.

Kosketustoiminnallisuus lisättiin käyttämällä *pointerInput*-parametria, jonka avulla elementin raahaamista voidaan hallita. Toiminnan alussa (*onDragStart*) voidaan nopeasti asettaa ehto siirtoelementin koon muuttumiselle sekä lisätä haptinen värinäreaktio, joka antaa käyttäjälle tunteen kosketuksen rekisteröitymisestä (Kuva 3).

Raahaamisen aikana (*onDrag*) saadaan *dragAmount*-property, joka sisältää X- ja Y-akselin uuden sijainnin siirron jälkeen. Tämä lisää siirtoelementin nykyiseen sijaintiin. Kosketustapahtuman eteenpäin siirtyminen muille UI-elementeille vältetään hyödyntämällä saatavaa *change*-propertyä.

Jotta osuma huomioi myös siirtoelementin leveys- ja korkeusarvot, luodaan erillinen *draggableBounds*-kehys, johon lisätään kyseiset arvot. Näitä rajoja hyödyntäen osuma voidaan helposti tarkistaa *Rect*-objektin *overlaps*-metodilla. Erillisen kehyksen luominen voitaisiin välttää, jos tapahtuma palauttaisi pelkän

siirron sijaan myös elementin mitat. Tällä hetkellä vastaava vaihtoehtoa ei kuitenkaan ole saatavilla. Kokonaisuudessaan aloitusnäkyvän toteutus Kotlin Multiplatformilla oli kohtalaisen yksinkertaista, ja virheenselvityksen tarve oli hyvin vähäistä.

7.1.2 Aloitusnäkyvän kehitys React Nativella

React Nativella aloitusnäkyvän toteutus aloitettiin ensin pelkästään Gesture Handler-kirjastoa hyödyntäen (Gesture Handler 2025). Tämä vaati useiden eri apufunktioiden luomista. Kosketuksen aikaiselle tapahtumalle määriteltiin oma apufunktio, jonka sisään lisättiin erillinen tapahtumakuuntelija (*listener*), jotta osuma voitiin tarkistaa jatkuvasti reaaliajassa.

Koska kirjasto ei tarjoa valmista ratkaisua osumien tarkasteluun, tälle luotiin erillinen apufunktio. Osumisen tarkasteluun liittyi myös ongelma: X- ja Y-sijainnit eivät aluksi huomioineet sovelluksen ylänavigoinnin tai statuspalkin viemää tilaa. Tämä korjattiin lisäämällä sekä nappeihin että liikutettavaan elementtiin referenssit, ja jokaiselle referenssille luotiin oma apufunktio alkupiirron aikaisen sijainnin tallentamiseen. Referenssi on viite UI-elementtiin, joka mahdollistaa sen suoran käsittelyn koodissa saatavilla olevien rajoitteiden puitteissa (React 2025). Sijainti saatiin apufunktioiden sisällä *measureInWindow*-metodilla selville ja tätä voitiin hyödyntää osumaa tarkasteltaessa.

Jotta elementin sijainti säilyi eikä aina palautunut lähtöpisteeseen, tarvittiin muuttujat sekä kertyneille pisteille X- ja Y-akselilla että tilannekohtaiselle siirron muutokselle. Lopullinen ratkaisu oli toimiva, mutta piirron aikana havaittiin huomattavaa flikkeröintiä, eli UI-komponentin tarkoituksetonta lyhytaikaista piirtymistä väärässä sijainnissa, mikä johti lisäselvityksiin paremman ratkaisun löytämiseksi.

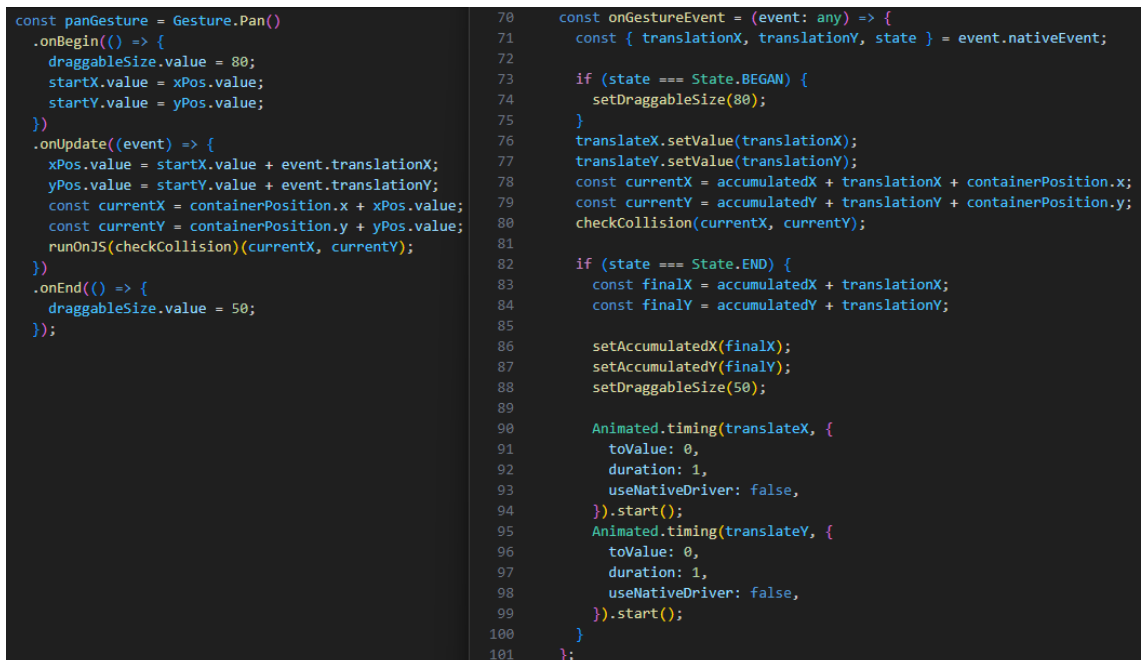
Selvityksen jälkeen päädyttiin muokkaamaan toteutusta hyödyntäen Reanimated-kirjastoa (Reanimated 2025), sillä se tarjoaa sulavamman ja suorituskykyisemmän animoinnin. Reanimated mahdollistaa animoinnin suorituksen suoraan UI-säikeellä, jolloin JavaScript-säikeen kuormitus vähenee

ja animaatiot pysyvät responsiivisina myös raskaammissa sovelluksissa (Reanimated 2025.)

Sen sijaan, että aiemman toteutuksen tapaan muutoksia olisi tallennettu stateihin ja referensseihin, ne säilöttiin nyt *useSharedValue*-hookin avulla. *SharedValue* toimii suoraan Reanimatedin sisäisessä animaatiojärjestelmässä, mikä mahdollistaa reaaliaikaiset muutokset ilman ylimääräisiä renderöintejä, parantaen suorituskykyä merkittävästi (Reanimated 2025)

Osumien tarkasteluun tarvittiin edelleen itse määritetty apufunktio, sillä kirjasto ei tarjoa siihen valmista ratkaisua. Lisäksi ylänavigoinnin alueet tuli yhä ottaa huomioon erillisillä referensseillä, joiden avulla elementtien todelliset sijainnit ruudulla saatiin mitattua.

Uusien muutosten myötä itse sovellus tuli sijoittaa *GestureHandlerRootView*-komponentin sisälle, mikä varmisti, että kosketustapahtumat havaitaan oikein. Siirrettävän elementin eleenkäsittelijä toteutettiin *Gesture.Pan*-hookilla, jossa *onUpdate*-funktio toimii UI-säikeellä, mikä parantaa animoinnin suorituskykyä (Kuva 4).



```
const panGesture = Gesture.Pan()
  .onBegin(() => {
    draggableSize.value = 80;
    startX.value = xPos.value;
    startY.value = yPos.value;
  })
  .onUpdate((event) => {
    xPos.value = startX.value + event.translationX;
    yPos.value = startY.value + event.translationY;
    const currentX = containerPosition.x + xPos.value;
    const currentY = containerPosition.y + yPos.value;
    runOnJS(checkCollision)(currentX, currentY);
  })
  .onEnd(() => {
    draggableSize.value = 50;
  });

const onGestureEvent = (event: any) => {
  const { translationX, translationY, state } = event.nativeEvent;

  if (state === State.BEGAN) {
    setDraggableSize(80);
  }

  translateX.setValue(translationX);
  translateY.setValue(translationY);
  const currentX = accumulatedX + translationX + containerPosition.x;
  const currentY = accumulatedY + translationY + containerPosition.y;
  checkCollision(currentX, currentY);

  if (state === State.END) {
    const finalX = accumulatedX + translationX;
    const finalY = accumulatedY + translationY;

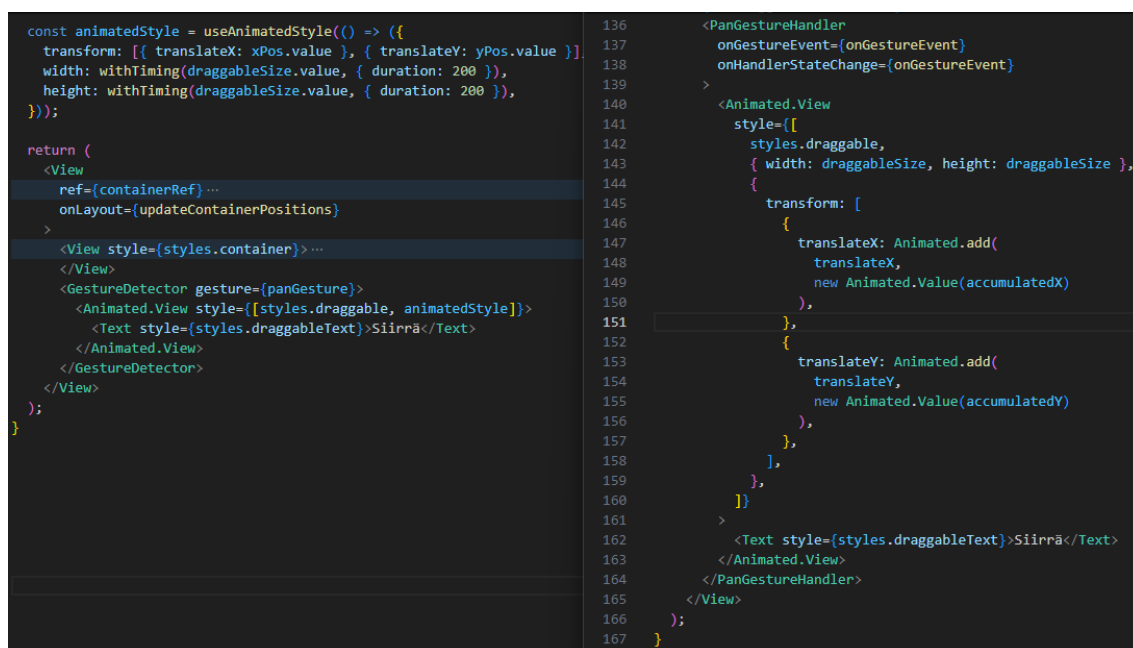
    setAccumulatedX(finalX);
    setAccumulatedY(finalY);
    setDraggableSize(50);

    Animated.timing(translateX, {
      toValue: 0,
      duration: 1,
      useNativeDriver: false,
    }).start();
    Animated.timing(translateY, {
      toValue: 0,
      duration: 1,
      useNativeDriver: false,
    }).start();
  }
};
```

Kuva 4. Reanimatedin (vas.) ja Gesture Handlerin (oik.) eleenkäsittelijät.

Koska *onUpdate*-funktio suoritetaan UI-säikeellä, osumatarkastelu tuli siirtää *runOnJS*-kutsun sisälle, jotta se toimisi JavaScript-säikeellä (Kuva 4).

Aiemmista toteutuksista poiketen elementin siirtämistä ei enää määritelty suoraan komponentin tyyliin, vaan sille luotiin erillinen *useAnimatedStyle*-hookilla määritelty animoitu tyyli, joka syötettiin suoraan elementin *style*-propille (Kuva 5). Tämän avulla pystyttiin myös mukauttamaan mahdollisia lisäefektejä, kuten animoinnin kestoa ja joustavuutta.



```

const animatedStyle = useAnimatedStyle(() => ({
  transform: [{ translateX: xPos.value }, { translateY: yPos.value }],
  width: withTiming(draggableSize.value, { duration: 200 }),
  height: withTiming(draggableSize.value, { duration: 200 }),
}));

return (
  <View
    ref={containerRef} ...
    onLayout={updateContainerPositions}
  >
    <View style={styles.container}> ...
    </View>
    <GestureDetector gesture={panGesture}>
      <Animated.View style={[styles.draggable, animatedStyle]}>
        <Text style={styles.draggableText}>Siirrä</Text>
      </Animated.View>
    </GestureDetector>
  </View>
);

```

```

136 <PanGestureHandler
137   onGestureEvent={onGestureEvent}
138   onHandlerStateChange={onGestureEvent}
139 >
140   <Animated.View
141     style={[
142       styles.draggable,
143       { width: draggableSize, height: draggableSize },
144       {
145         transform: [
146           {
147             translateX: Animated.add(
148               translateX,
149               new Animated.Value(accumulatedX)
150             ),
151           },
152           {
153             translateY: Animated.add(
154               translateY,
155               new Animated.Value(accumulatedY)
156             ),
157           },
158         ],
159       },
160     ]}
161   >
162     <Text style={styles.draggableText}>Siirrä</Text>
163   </Animated.View>
164 </PanGestureHandler>
165 </View>
166 );
167 }

```

Kuva 5. Reanimatedin (vas.) ja Gesture Handlerin (oik.) siirtomäärittysten eroavaisuudet.

Lopullinen toteutus oli verrattain helppo saavuttaa, sillä Reanimated-kirjasto tarjosi erinomaiset dokumentaatiot sen käyttöön. Lisäksi toteutus toimi erittäin sulavasti, eikä flikkeröintiä havaittu emuloidulla laitteella.

7.1.3 Aloitusnäytön kehityksen vertailu

Aloitusnäytön toteuttaminen oli huomattavasti nopeampaa Kotlin Multiplatformilla, sillä tekoäly pystyi tarjoamaan heti toimivat ratkaisut haluttuihin ominaisuuksiin ilman merkittävää virheenselvittelyä. React Nativella sen sijaan

kehitys vaati enemmän hienosäätöä ja virheenselvitystä, erityisesti koska elementtien sijainnit eivät vastanneet todellista asemaansa ruudulla suhteessa muihin käyttöliittymäkomponentteihin.

Kotlin Multiplatformin tarjoamat sisäänrakennetut toiminnot, kuten elementtien sijainnin käsittely ja osuman tarkastelu, hoituivat taustalla, mikä vähensi manuaalista määrittelyä. React Nativen tapauksessa nämä toiminnot vaativat erillisiä apufunktioita sekä kumulatiivisten sijaintimuutosten erillistä tallentamista. Tämä lisäsi toteutuksen monimutkaisuutta ja toi mukanaan enemmän potentiaalisia virhetilanteita.

Kotlin Multiplatformin valmiiden toimintojen ansiosta toteutus pysyi selkeänä ja koodirivien määrä huomattavasti pienempänä. Tässä tapauksessa Kotlin Multiplatform-toteutuksessa ei tarvittu yhtäkään erillistä apufunktiota, kun taas React Nativen Reanimated-versiossa niitä oli perustellusti kaksi, yksi elementtien sijainnin määrittelyyn ja toinen osumien tarkasteluun.

Syntaksin rakenteen kannalta toteutukset eivät olleet merkittävästi toisistaan poikkeavia, mutta koodirivien määrässä oli huomattava ero.

Jetpack Compose-toteutuksessa koodirivien määrä oli selkeästi alhaisin, kun taas Gesture Handlerilla koodia oli lähes kaksinkertainen määrä siihen nähden (Taulukko 6). Luvuista on poistettu kirjastojen importit, muu kosketustoimintaan liittymätön koodi sekä erilliset tyylimääritykset.

Taulukko 6. Aloitusnäkyvän toteutunut kosketuskeskeinen koodirivimäärä.

Käytetty kirjasto	Rivimäärä
Gesture Handler	157
Reanimated	112
Jetpack Compose	79

Visuaalisesti sekä Jetpack Compose- että Reanimated-toteutukset toimivat sulavasti ja ilman havaittavia virheitä emuloidulla laitteella. Kosketukseen reagointi oli viiveetöntä ja animaatiot näyttivät luonnollisilta. Gesture

Handler-toteutuksessa sen sijaan esiintyi havaittavaa flikkeröintiä, mikä heikensi käyttökokemusta. Tästä syystä seuraavissa toteutuksissa keskitytään Reanimatedin käyttöön, mikäli sillä voidaan toteuttaa kaikki tarvittavat ominaisuudet.

Kaikilla toteutustavoilla halutut ominaisuudet olivat täysin mahdollisia, mutta React Nativen kohdalla tarvittiin odotettua enemmän lisämääryityksiä sekä osumien että sijainnin hallintaan.

Dokumentaation ja kehittäjäkokemuksen vertailu

Reanimatedin viralliset dokumentaatiot tekivät sen käyttöönotosta erittäin helppoa (Reanimated 2025). Ohjeet ja esimerkit olivat lyhyitä, havainnollistavia ja aloittelijaystävällisiä, mikä nopeutti kehitysprosessia. Jetpack Compose (Android Developers 2025e) dokumentaatio Android Developers-sivustolla on erittäin kattava, mutta tiedon löytäminen vaati enemmän aikaa ja tuntui kuormittavammalta.

Tekoälyn (OpenAI 2025a) hyödyntäminen Kotlin Multiplatformin kanssa oli suoraviivaisempaa, sillä virheenselvitystä ei juuri tarvittu. React Nativen kohdalla tekoälystä saatu apu vaati huomattavasti enemmän omaa tarkistusta ja virheenkorjausta, mikä lisäsi kehitysaikaa merkittävästi.

Molemmissa ympäristöissä muutosten tekeminen oli intuitiivista, mutta uusien kirjastojen ominaisuuksien hyödyntäminen edellytti verkkodokumentaation käyttöä. Reanimatedin IDE-integroitu dokumentaatio tarjoaa suorat linkit esimerkkikoodiin ja ohjeisiin sen sijaan, että se listaisi yksityiskohtaisia komponenttitietoja. Kotlin Multiplatformin kehitysympäristö pystyy jossain määrin täydentämään tarvittavia konfiguraatioita uusien ominaisuuksien käyttöönotossa ilman ulkoisia resursseja, mutta se ei ole täysin aloittelijaystävällinen.

7.2 Piirtotoiminnallisuus

Piirtotoiminnallisuus sen näkymässä tukee enintään kolmea samanaikaista kosketusta. Käyttäjä voi piirtää yhdellä, kahdella tai kolmella sormella tai muokata viivan väriä ja paksuutta samalla, kun muut sormet voivat olla käytössä tai lepotilassa. Kosketusten tulee toimia itsenäisesti, jotta yhden piirron päättyessä muut piirtotapahtumat eivät keskeydy. Värivaihtoehtoja on kaksi, esitettyinä ympyrämaisina nappeina, ja viivan paksuutta säädetään säätöliukua painamalla.



Kuva 6. Piirtosivun yleisilme.

Toteutuksen keskeinen seurantatavoite on useiden samanaikaisten kosketusten hallinta, kun taas visuaalinen ilme ja käyttöliittymä voivat mukautua tarpeen mukaan.

7.2.1 Piirtotoiminnallisuus Kotlin Multiplatformilla

Piirtosivu Kotlin Multiplatform-sovelluksessa toteutettiin Jetpack Compose-kirjastolla ja sen *Canvas*-komponentilla, joka tarjoaa tehokkaan tavan piirtää vektoripohjaista grafiikkaa ja mahdollistaa reaaliaikaisen piirtämisen suoraan käyttöliittymässä (Android Developers 2025b). Canvas toimii piirtopintana, johon voidaan piirtää viivoja, muotoja ja muita visuaalisia elementtejä käyttäjän syötteiden perusteella.

```
Canvas(
    modifier = Modifier
        .fillMaxSize()
        .pointerInput(Unit) {...}
) {
    paths.forEach { path ->
        drawPath(path, color = selectedColor, style = Stroke(width = 5f))
    }

    activePaths.values.forEach { path ->
        drawPath(path, color = selectedColor, style = Stroke(width = 5f))
    }
}
```

Kuva 7. Canvaksen käyttö ja piirtopolkujen renderöinti Kotlin Multiplatformilla.

Toteutuksen aikana ilmeni erityisesti alkuvaiheessa haasteita, sillä tekoälyltä (OpenAI 2025a) saatu tieto tarjosi sopimattoman eleidenhallinnan funktion (*awaitEachGesture*) kosketusten tunnistamiseen, minkä seurauksena piirretty jana ei näkynyt ruudulla lainkaan. Tarkastelemalla liikkuvaa dataa selvisi, että kyseinen funktio pystyy tarjoamaan vain tuoreimman eleen sijaintitiedot kerrallaan, jolloin se ei soveltunut haluttuun monikosketuspiirtoon. Toinen kehityksessä esiin noussut haaste liittyi Jetpack Composen tilamuuttujien hallintaan. Vaikka muuttujat oli kääritty *mutableStateOf*-säilöön ja olivat siten muokattavissa, pelkkä objektin sisällön päivittäminen ei käynnistänyt uutta renderöintiä.

Compose havaitsee tilan muuttuneen vain, jos muuttujaan asetetaan kokonaan uusi arvo (Android Developers 2025g). Tämän vuoksi aktiivisten polkujen väliaikaisesta säilöstä täytyi luoda aina uusi olio (Kuva 8), joka sitten korvattiin tilamuuttujassa. Näin jokainen muutos rekisteröitiin, ja piirtämisen aikana syntyvät viivat saatiin näkyviin reaaliajassa ilman viivettä.

```
if (existingPath == null) {  
    existingPath = Path().apply {  
        moveTo(position.x, position.y)  
    }  
}  
val newPath = Path().apply {  
    addPath(existingPath)  
    lineTo(position.x, position.y)  
}  
  
updatedPaths[pointerId] = newPath
```

Kuva 8. Uuden polun (*Path*) asettaminen vanhan päivityksen sijasta.

Tämän jälkeen tehtävää lähdettiin ratkaisemaan vaiheittain. Ensimmäisessä toteutuksessa luotiin yksinkertainen piirtomekanismi, joka salli vain yhden sormen piirron kerrallaan. Seuraavaksi toteutus laajennettiin tukemaan useampaa yhtäaikaista kosketusta hyödyntämällä *awaitPointerEventScope*-funktia (Kuva 9). Tämä funktio mahdollistaa kaikkien aktiivisten kosketuspisteiden jatkuvan seurannan ilman, että yhden sormen kosketus peruisi toisen.

```

.pointerInput(Unit) {
    awaitPointerEventScope {
        while (true) {
            val event = awaitPointerEvent()
            val activePointers = event.changes.filter { it.pressed }

            if (activePointers.size > 3) continue

            val updatedPaths = activePaths.toMutableMap()

            for (touch in activePointers) {
                val pointerId = touch.id
                val position = touch.position

                if (handleColorButtonPress(position, buttonSize, densi

                if (touch.pressed) {

```

Kuva 9. Kotlin Multiplatformin tapahtumien kuuntelijan toiminnallisuuksia.

Jokaiselle kosketukselle tallennettiin yksilöllinen tunniste (ID), jonka avulla voitiin seurata erillisiä kosketuspisteitä (Kuva 9). Tämän ansiosta pystyttiin arvioimaan, tulisiko aloittaa uusi piirtopolku vai jatkaa jo olemassa olevaa. Piirtomekanismi hyödynsi kahta erillistä säilöä: aktiivisia piirtopolkuja, jotka sisälsivät parhaillaan piirrettävät viivat, sekä valmiita piirtopolkuja, joihin siirrettiin jo piirretyt viivat kosketuksen päättyessä.

Kosketusten reagoimista keskenään, ei tarvinnut erikseen konfiguroida, vaan ne toimivat automaattisesti kyseisellä ratkaisulla niin, ettei yksittäinen nostettu tai laskettu kosketus vaikuttanut muihin.

Kun käyttäjä nosti sormen näytöltä, siihen liittyvä piirtopolku poistettiin aktiivisten joukosta ja tallennettiin lopullisten piirtopolkujen säilöön. Tämä mahdollisti sujuvan ja monikosketusta tukevan piirtotoiminnon ilman, että keskeneräiset ja valmiit viivat sekoittuivat keskenään.

Koodirivit olivat suhteellisen lyhyitä, eikä erillisille apufunktioille koettu tarvetta, paitsi värinvaihdon ja liukurin kosketuksen arvioinnissa. Värinapeille ja liukurille määritettiin oma alueensa, johon osuessa piirtopolut keskeytyivät. Tämä varmistettiin siten, että piirtomekanismi tarkasti jokaisen kosketuksen sijainnin ja esti piirtämisen, mikäli kosketus tapahtui näillä alueilla. Näin vältettiin tilanteet, joissa viivat jatkuisivat painikkeiden tai säätimien yli, ja varmistettiin käyttöliittymän sujuva toiminta.

Muutosten tekeminen tuntui aluksi hieman haastavalta, sillä virallisissa dokumentaatioissa ei ollut yksinkertaisia esimerkkejä funktioiden käytöstä, ja niiden ominaisuuksien selvittäminen vaati usein usean eri osan dokumentoinnin läpikäymistä. Kuitenkin kun jo tutuksi tulleet toiminnot saatiin käyttöön, työskentely muuttui huomattavasti sujuvammaksi. Syntaksin ollessa selkeää ja kohtalaisen lyhyttä, se ei aiheuttanut kuormituksen tunnetta, vaan mahdollisti sujuvan tilannekohtaisen kehityksen.

7.2.2 Piirtotoiminnallisuus React Nativella

Piirtosivu React Native -sovelluksessa toteutettiin hyödyntämällä yhdessä Skia-, Reanimated- ja Gesture Handler-kirjastoja. Skia valittiin, koska se tarjoaa tehokkaan tuen reaaliaikaiselle piirtämiselle. Skia huolehtii GPU-kiihdytetystä renderöinnistä, mikä pitää viivojen piirrot sujuvina eikä aiheuta frame droppeja, vaikka pisteiden määrä kasvaisikin merkittävästi. Skia tukee myös vektoriviivojen tasoitusta suoraan renderöintimoottorissaan, mikä edesauttaa piirrosjäljen laatua (React Native Skia 2025). Vastaavassa tilanteessa Reanimated ei ole optimoitu vapaamuotoiseen piirtoon, vaan se soveltuu paremmin komponenttien animointiin ja raskaassa käytössä se voi kuormittaa, etenkin jos laite tukee vain 60 FPS:n nopeutta (Reanimated 2025).

Toteutuksen aikana ilmeni useampia haasteita, jotka vaativat manuaalista selvitystä. Esimerkiksi *onTouch*-hookeissa tarvittiin *runOnJS*-wrappauksen käyttöä, jotta React-tilan päivitykset suoriutuisivat oikein. Alustavasti käytetyt hookit (*onBegin*, *onUpdate* ja *onEnd*) tukivat vain yhden sormen seuranta,

niiden näyttäessä sijaintien keskiarvo useamman kosketuksen tapahtumissa. Tämän seurauksena hookit vaihdettiin *onTouches*-tapahtumiin, jotka pystyvät raportoimaan jokaisen kosketuksen sijainnin.

Ohjaamalla keskustelua tekoälyn kanssa saatiin rajattua virhekohtia, sekä päädyttiin kokeilemaan vaihtoehtoisia toiminnallisuuksia niiden ratkaisemiseksi. Tämä johti nopeaan edistymiseen kehityksessä.

Aktiiviselle piirrolle ja aiemmin tallennetuille piirroksille luotiin omat muuttujasäilöt. Aktiivinen piirros renderöitiin reaaliajassa ja tallennettiin erilliseen muuttujaan, kunnes kosketus päättyi, jolloin siihen liittyvä polku siirrettiin pysyvämpään säilöön. Kaikki visuaaliset elementit renderöitiin Skian *Canvas*-komponenttien sisällä (Kuva 10).

```
<Canvas style={{ flex: 1 }}>
  {paths.map(({ path, color, width }, index) => (
    <Path
      key={index}
      path={path}
      color={strokeColor}
      style="stroke"
      strokeWidth={strokeWidth}
    />
  ))}
  {Object.values(activePaths).map((path, index) => (
    <Path
      key={`active-${index}`}
      path={path as any}
      color={strokeColor}
      style="stroke"
      strokeWidth={strokeWidth}
    />
  ))}
</Canvas>
```

Kuva 10. Canvaksen käyttö ja piirtopolkujen renderöinti React Nativella.

Selkeyden vuoksi toteutukseen lisättiin erilliset apufunktiot start-, end-, update- ja lisätoiminnallisuudelle, jolloin kaikkiaan apufunktioita oli neljä (Kuva 11).

```
const panGesture = Gesture.Pan()
  .maxPointers(3)
  .onTouchesDown((event) => {
    runOnJS(handleTouch)(event);
    runOnJS(handleStart)(event);
  })
  .onTouchesMove((event) => {
    runOnJS(handleUpdate)(event);
  })
  .onTouchesUp((event) => {
    runOnJS(handleEnd)(event);
  });
```

Kuva 11. Apufunktiot ja rakenne piirtogesturessa.

Lisätoiminnan apufunktiossa tarkasteltiin, onko mikään kosketustapahtuma värejä vastaavien nappien alueella, tai osuivatko ne säätöliukuun. Muut apufunktiot konfiguroitiin tunnistamaan kosketukset, jotka alkoivat tai päättyivät toisten kosketusten ollessa yhä aktiivisia (Kuva 12).

```
const handleEnd = (event) => {
  setPaths((prevPaths) => [
    ...prevPaths,
    ...event.changedTouches
      .map((touch) => ({
        path: activePaths[touch.id], // Siirretään vain nostettuja viivoja
      })))
      .filter(({ path }) => path !== undefined), // Validien polkujen varmistus
  ]);

  setActivePaths((prevPaths) => {
    const updatedPaths = { ...prevPaths };
    event.changedTouches.forEach((touch) => {
      delete updatedPaths[touch.id]; // Poistetaan vain nostettu viiva
    });
    return updatedPaths; // Säilytetään muut aktiiviset viivat
  });
};
```

Kuva 12. Piirtomuuttujien hallinta ja manipulointi apufunktiossa.

Muutosten tekeminen oli intuitiivista ja helppoa olemassa olevan materiaalin pohjalta. Uusien toimintojen lisääminen saattaisi kuitenkin vaatia lisäselvitystä ja verkkoselailua. Testaaminen edellytti fyysistä puhelinta, sillä emulaattori tunnisti

vain yhden kosketuksen kerrallaan. Fyysisellä laitteella testaus sujui kuitenkin vaivattomasti Expo Go:n avulla.

7.2.3 Piirtotoiminnallisuuden vertailu

Piirtonäkymän toteutus jakautui molempien vertailtavien teknologioiden välillä tasapuolisesti sekä työmäärän että virheenselvityksen osalta. Tekoälyn tarjoamat ratkaisut eivät olleet täysin yhteensopivia haluttujen toimintojen kanssa, mikä johti kohtuulliseen määrään virheiden analysointia ja korjaamista molemmissa tapauksissa.

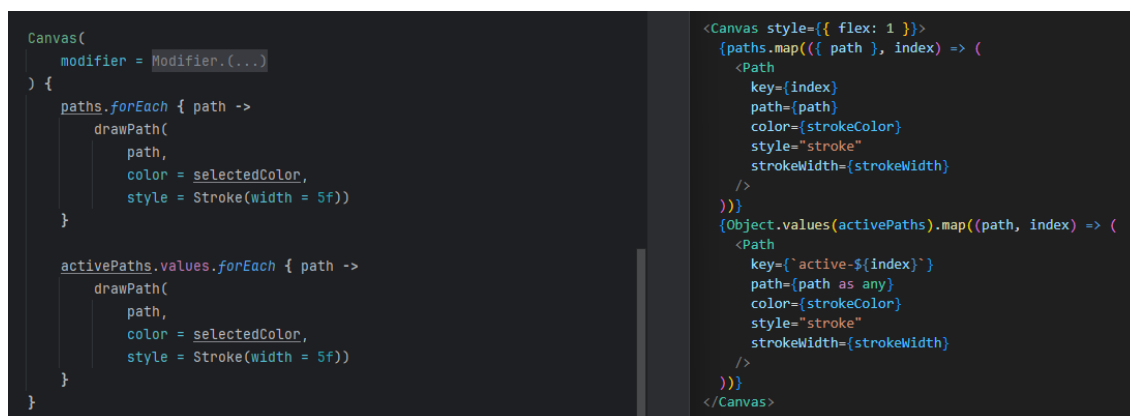
Ei React Native eikä Kotlin Multiplatformissa ollut valmista ratkaisua törmäystarkasteluun yksittäisen pisteen ja kehyksen välillä, vaan tämä toiminnallisuus oli toteutettava manuaalisesti.

React Nativessa tarvittiin enemmän apufunktioita, sillä sen eleidentunnistaja sisälsi erilliset *hooks* kosketuksen aloitukselle, päivitykselle ja lopetukselle. Vaikka tämä lisäsi koodin määrää, se teki tapahtumien hallinnasta selkeämpää ja helpommin muokattavaa.

Kotlin Multiplatformin tapauksessa monimutkaisuutta lisäsi tarve luoda uusi olio jokaiselle piirron polulle aina päivityksen yhteydessä. Ilman tätä toimenpidettä graafinen muutos ei renderöitynyt reaaliajassa, vaan vasta kosketuksen päättyessä. Tämä erillinen lisävaatimus saattoi kasvattaa virhetilanteiden riskiä, jos sen toteutus jossain vaiheessa unohtuisi. React Nativen puolella tavanomainen tilan päivitys settereillä riitti vastaavan lopputuloksen saavuttamiseen.

Molempien teknologioiden piirtoalustojen syntaksi oli rakenteeltaan ja ominaisuuksiltaan hyvin samankaltainen. Suurimpana erona oli se, että React Nativessa taustalla käytettiin erillistä merkintätapaa, kuten hakasulkeita ja muita JSX-rakenteita (Kuva 13). Lisäksi, kun React Nativen tapauksessa kosketustoiminnallisuus määritellään koko näkymän kattavassa

GestureDetector-komponentissa, Kotlinissa se asetetaan suoraan Canvas-komponentin pointerInput-kenttään.



Kuva 13. Piirtoalustojen syntaksin vertailu Kotlin Multiplatformin (vasemmalla) ja React Nativeen (oikealla) välillä.

Kotlinin Jetpack Compose-kirjaston tapauksessa kosketustoimintojen toteuttaminen vaati hieman vähemmän koodirivejä verrattuna React Nativeen, jossa hyödynnettiin Gesture Handleria ja Skiaa (Taulukko 7). Vaikka ero ei ole merkittävä, se saattaa viitata siihen, että Kotlinilla laajempien sovellusten toteutus voi vaatia vähemmän koodirivejä.

Taulukko 7. Piirtonäkymän toteutunut kosketuskeskeinen koodirivimäärä.

Käytetyt kirjastot	Rivimäärä
Gesture Handler, Skia	30
Jetpack Compose	41

Dokumentaation ja kehittäjäkokemuksen vertailu

React Native Skian ja Jetpack Compose Canvasin virallisissa dokumentaatioissa ei ollut suoraa ohjeistusta piirtotoiminnallisuuden toteuttamiseen. Skian dokumentaatio sisälsi selkeämmin jäsenneltyä ja käytännönläheistä materiaalia, minkä perusteella tarvittavan toiminnallisuuden

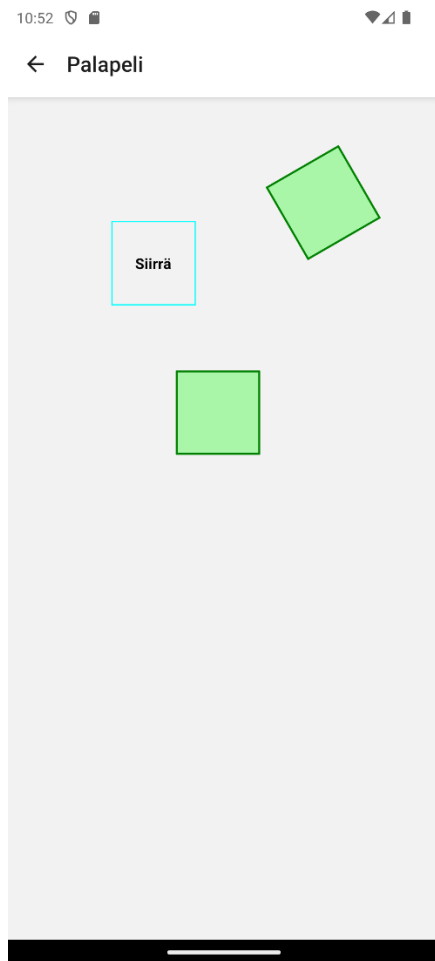
pystyi päätelmään. Lisäksi Skiasta oli saatavilla useita kolmannen osapuolten laatimia ohjeita, mitkä olivat selkeitä ja helposti ymmärrettäviä.

Jetpack Compose Canvasin kohdalla vastaavaa materiaalia oli vähemmän saatavilla, eikä erityisesti monen sormen samanaikaista piirtämistä käsitteleviä artikkeleita löytynyt helposti.

Kummassakin tapauksessa toiminnallisuudet olivat täysin toteutettavissa ja sujuvia käyttää kosketusnäytöllä. Intuiivisten muutosten tekeminen oli kummassakin tapauksessa haastavaa uusien funktioiden käyttöönoton yhteydessä. Kun toimintoihin oli perehdytty, muutosten tekeminen helpottui, sillä syntaksi oli itsessään helposti ymmärrettävissä. Lähtökohtaisesti funktioihin oli tutustuttava verkossa saatavilla olevan materiaalin avulla ennen niiden tehokasta käyttöönottoa. Merkittävin ero oli React Nativen Gesture Handlerin tapa erotella selkeästi eri tapahtumat, aloitus, päivitys ja lopetus, sekä se, ettei piirron aikana ollut tarpeen luoda uusia olioita reaaliaikaisen päivityksen takaamiseksi.

7.3 Palapeliteoiminnallisuus

Palapeli-sivu sisältää siirrettävän nelikulmaisen elementin, jonka tarkoituksena on asettua paikoilleen ennalta määriteltuihin nelikulmaisiin kehyksiin (Kuva 14). Liikuteltavaa elementtiä voidaan samanaikaisesti siirtää koskettamalla suoraan sen päältä ja toisella sormella pyörittämällä mistä tahansa ruudun sijainnista. Rotaatio lasketaan kahden sormen muodostamasta kulmasta, jossa vain jälkimmäisen sormen liike aiheuttaa rotaation.



Kuva 14. Palapelisivun yleisilme.

Snap-toiminnallisuus aktivoituu automaattisesti, kun siirrettävä elementti on tarpeeksi lähellä jotakin kehyksistä sekä sen kulma on riittävän lähellä kehyksen kulmaa. Tällöin elementti "napautetaan kiinni" kehyksen paikalle. Toteutuksessa on kaksi kehystä, joista toinen on valmiiksi pyöräytetty, jotta voidaan testata myös rotaation huomioiminen snap-toiminnossa (Kuva 14).

Toteutuksen keskeinen seurantakohde on elementin rotaation toteutus vapaavalinnaisesta sijainnista, samalla kun sitä siirretään. Lisäksi arvioidaan, kuinka hyvin elementtien *transform*-ominaisuuksia (sijainti, koko, rotaatio) voidaan hallita niin, että ne ovat vertailtavissa ja pinottavissa toistensa päälle. Visuaalinen ilme ja käyttöliittymä voivat mukautua tarpeen mukaan.

7.3.1 Palojen yhtäaikainen siirtäminen ja rotaatio Kotlin Multiplatformilla

Palapelisivu toteutettiin käyttäen Jetpack Composea, sillä tarvetta erillisille kirjastoille ei esiintynyt.

Toteutus eteni ajoittain hitaasti, sillä joitakin ongelmia oli aluksi vaikea jäljittää, ja osa toiminnallisuuksista vaati paljon manuaalista työtä, erityisesti kosketus- ja rotaatiohallinnan osalta. Toisaalta aiemmin tehdyt osuudet autoivat ymmärtämään esimerkiksi *pointer*-järjestelmän toimintaa. Eräs selvittämättä jäänyt ongelma oli, että tietyt elementit eivät reagoineet rotaatioon odotetulla tavalla, kun ne oli asetettu komponenttien sisälle *child*-elementteinä.

Toteutuksen aikana kokeiltiin myös Compose Gesture Detector-kirjaston tarjoamia valmiita vaihtoehtoja kuten *detectTapGestures* ja *detectTransformGestures*, jotka tarjoavat sisäänrakennettua tukea rotaatiolle ja skaalaukselle. Näistä kuitenkin luovuttiin, sillä ne eivät tarjonneet riittävää kontrollia yksittäisten sormien liikkeisiin halutunlaisissa tilanteissa, joissa yksi sormi siirtää elementtiä ja toinen kääntää sitä ruudun eri kohdasta. Lopullinen ratkaisu perustui *awaitPointerEventScope*-menetelmän käyttöön, jonka avulla saatiin tarkka hallinta yksittäisiin kosketustapahtumiin ja niiden kulkuun.

Tilamuuttujien hallintaan käytettiin *mutableStateOf*-muuttujia, joiden avulla voitiin reaktiivisesti päivittää näkymää sijainnin, kulman ja muiden muuttujien päivittyessä. Tämä on tärkeää Compose-ympäristössä, jossa käyttöliittymä uudelleen piirtyy automaattisesti tilan muuttuessa.

Rotaatio syötettiin elementtiin *graphicsLayer*-ominaisuudella (Kuva 15) *rotate*-ominaisuuden sijaan, minkä valinta perustuu sen modernimpaan suosioon, tarkempaan keskikohdan, sekä loogisen sijainnin asetukseen (Android Developers 2025c). Koska laskutoimitukset oli toteutettu radiaaneilla, ne tuli kääntää asteiksi elementtejä varten.

```

snapTargets.forEach { target ->
    Surface(
        modifier = Modifier
            .offset { IntOffset(target.position.x.roundToInt(), target.position.y.roundToInt()) }
            .size(100.dp)
            .graphicsLayer(rotationZ = (target.rotation * 180 / Math.PI).toFloat()),
        color = Color.Red
    ) {}
}

Surface(
    modifier = Modifier
        .offset { IntOffset(offset.x.roundToInt(), offset.y.roundToInt()) }
        .size(100.dp, 100.dp) // Define size for the rectangle
        .graphicsLayer(rotationZ = (rotation * 180 / Math.PI).toFloat()),
    color = Color.Blue
) {
}

```

Kuva 15. Elementtien renderöinti palapelisivulla (Kotlin Multiplatform).

Snap-toiminto toteutettiin siten, että kun käyttäjä vapautti kaikki sormensa näytöltä, tarkistettiin onko siirrettävä elementti lähellä jotakin ennalta määriteltyä kohdetta. Jokaisella snap-kohteella oli määritelty sekä sijainti että kiertosuunta. Mikäli siirrettävä elementti oli tarpeeksi lähellä jotakin kohdetta ja sen rotaatio lähellä kohteen kulmaa, elementti "napsahti" kohteen päälle sekä sijainnin että kulman osalta. Tämä vaati kulman käsittelyä asteina sekä absoluuttisten arvojen laskentaa, jotta myös negatiiviset kulmat tulkittiin oikein.

Syntaksi osoittautui selkeäksi ja yksinkertaiseksi. Haluttuihin ominaisuuksiin päästiin käsiksi helposti, ja arvoja voitiin suoraan hallita tilamuuttujien kautta. Ainoa selkeästi monimutkaisempi osa-alue liittyi matemaattisiin laskutoimituksiin, erityisesti rotaation ja vektorien käsittelyyn. Näiden kaavojen pituuden vuoksi oli perusteltua eriyttää ne omiin apufunktioihinsa, mikä osaltaan myös selkeytti pääasiallista logiikkaa.

Yksinkertaisen syntaksin ansiosta muutosten tekeminen oli pääosin intuitiivista, erityisesti tilanteissa, joissa toimittiin tuttujen funktioiden rajoissa. Uusien toimintojen lisääminen sen sijaan edellytti usein dokumentaation tai lisämateriaalien tutkimista. Lisäksi lievää haastetta muutosten tekoon toi selvittämätön ongelma, jossa jotkut *child*-elementit eivät reagoineet rotaatioon

odotetusti. Tämän seurauksena jouduttiin kokeilemaan useita sisäänrakennettuja komponentteja, jotta haluttu lopputulos saavutettiin.

Verkosta löytyvä dokumentaatio tarjosi hyvin perustason esimerkkejä, mutta mitä monimutkaisempia ratkaisuja yritettiin toteuttaa, tai mitä epäselvempiin konsepteihin haettiin vastauksia, sitä enemmän ongelmat jäivät itsenäisesti ratkaistaviksi. Usein esimerkiksi Stack Overflowsta löytyi kysymyksiä ilman vastauksia ja tämä saattaa myös osaltaan selittää, miksi tekoälyavusteiset ratkaisut painottuivat raakojen *pointer*-tapahtumien hyödyntämiseen valmiiden *gesture*-käsittelijöiden sijaan, sekä miksi useat toiminnot, kuten rotaation käsittely, toteutettiin manuaalisesti vektorilaskentaa hyödyntäen.

Testaus vaati fyysisen älypuhelimien yhdistämisen, sillä emuloidussa laitteessa kahden sormen kosketus ei onnistu, minkä vuoksi rotaatioita ei olisi voitu tarkastella.

7.3.2 Palojen yhtäaikainen siirtäminen ja rotaatio React Nativella

Palapelisivu React Native -sovelluksessa toteutettiin hyödyntämällä sekä *Reanimated*-että *Gesture Handler*-kirjastoja, sillä tarvetta muille ulkoisille kirjastoille ei esiintynyt.

Toteutuksessa tarvittiin soveltavaa otetta ja virheiden selvittelyä, mutta tässä vaiheessa suurin osa esiin tulleista ongelmista oli jo entuudestaan tuttuja, eikä niiden ratkaiseminen vaatinut suurta työtä.

Alkuvaiheessa rotaation toteutusta lähdettiin rakentamaan *Gesture.Rotate*-funktion avulla, mutta pian kävi ilmi, ettei se soveltunut haluttuun käyttötarkoitukseen. Tavoitteena oli mahdollistaa elementin liikuttaminen ja kiertäminen samanaikaisesti, mutta myös erikseen ilman rotaatiota. *Gesture.Rotate* edellyttää, että molemmat sormet ovat aktiivisia eleen alusta alkaen, ja se muuttaa elementin kulmaa ainoastaan sormien välisen kulman perusteella (*Gesture Handler* 2025). Tämä ei vastannut haluttua toiminnallisuutta, joten päädyttiin käyttämään *onTouches*-funktioita, jotka

mahdollistavat useamman samanaikaisen tapahtuman seuraamisen ja antavat täydellisen hallinnan kosketusdatan käsittelyyn.

Toteutuksessa käytettiin *useSharedValue*-muuttujia dynaamisten arvojen, kuten sijainnin, koon ja kulman tallentamiseen, koska ne mahdollistavat sujuvan animaation ja Reanimatedin kanssa toimivan tehokkaan tilan hallinnan.

Rotaation laskenta toteutettiin manuaalisesti vektorilaskennan avulla, määrittämällä vektorit elementin keskikohdasta toiseen kosketuspisteeseen ja laskemalla kulmamuuтокset niiden perusteella ja syötettiin elementteihin Reanimatedin *useAnimatedStyle*ä hyödyntäen (Kuva 16). Yksinkertaisempi lähestymistapa manuaalisen laskennan sijaan olisi voitu toteuttaa olemassa olevilla kirjastoilla mutta niistä ei löydetty tukea monimutkaiselle kiertämiselle tai rotaatioasentojen huomioimiselle.

```
{snapTargets.map((target, index) => {
  const snapStyle = getSnapStyle( target, index, snappedTargetIndex, elementSize );
  return <Animated.View key={index} style={snapStyle} />;
})}

<GestureDetector gesture={panAndRotateGesture}>
  <Animated.View style={[styles.draggable, animatedStyle]}>
    <Text style={styles.draggableText}>Siirrä</Text>
  </Animated.View>
</GestureDetector>
```

Kuva 16. Palapelinäkymän piirtoelementit (React Native).

Palapelimainen toiminnallisuus puolestaan toteutettiin määrittelemällä snap-alueet, joihin elementti kiinnittyy automaattisesti, jos se on tarpeeksi lähellä ja orientaatio on sopiva. Tämä toteutettiin vertaamalla elementin ja kohdealueen keskikohtien välistä etäisyyttä sekä tarkastelemalla elementin kulmaa neljässä mahdollisessa käännoisasennossa, jotta se osuu tarkasti paikoilleen. Snäppäys tapahtuu animoidusti käyttäen *withTiming*-funktiota, mikä muodostaa sulavan siirtymän.

Syntaksi itsessään oli yksinkertaista ja helposti ymmärrettävää. Tässä vaiheessa käytetyt ominaisuudet olivat jo ennestään tuttuja, mikä osaltaan vaikutti arvioon niiden selkeydestä. Kompleksisuutta toi kuitenkin manuaalinen

vektorilaskenta, joka voi vaikeuttaa kehitystyötä erityisesti rotaatioiden käsittelyssä, mikäli kehittäjällä ei ole tarvittavaa matemaattista osaamista.

Matemaattisten laskelmien vuoksi koodin jakaminen erillisiin apufunktioihin oli perusteltua, sillä se paransi luettavuutta ja ylläpidettävyyttä.

Muutosten tekeminen oli intuitiivista, sillä toiminnallisuuden toteuttamiseen oli tarjolla paljon vapautta. Testaaminen vaati kuitenkin fyysisen puhelimen, sillä emulaattori tuki vain yhtä kosketusta kerrallaan. Fyysisellä laitteella testaus sujui kuitenkin vaivattomasti Expo Go-sovelluksen avulla.

7.3.3 Palapelitoiminnallisuuden vertailu

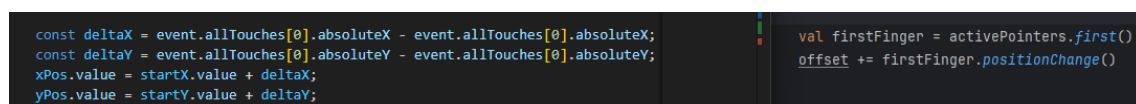
Palapelinäkymän toteutus molemmilla vertailtavilla teknologioilla vaati virheenselvitystä sekä toistuvaa iterointia ja mukauttamista. Kummassakin tapauksessa rotaation kulman laskenta jouduttiin toteuttamaan manuaalisesti, sillä teknologiat eivät tarjonneet suoraan sisäänrakennettuja toimintoja halutunlaiseen, vapaamuotoiseen rotaation hallintaan. Vaikka molemmista teknologioista löytyy valmiita gesture-funktioita rotaation käsittelemiseksi, ne edellyttävät eleen alkavan kahden sormen kosketuksella, eikä niitä ollut mahdollista mukauttaa tarpeeseen, jossa elementtiä haluttiin siirtää yhdellä sormella ja samalla kiertää toisella.

Rotaation laskennan, snappauksen ja renderöinnin logiikka oli molemmissa tapauksissa pitkälti vastaavanlainen. Erot syntyivät lähinnä siitä, miten itse laskenta toteutettiin, sekä siitä, millaisia ominaisuuksia ja dataa sisäänrakennetut funktiot eri ympäristöissä tarjoavat. Myös syntaksin muoto ja kielikohtaiset tilamuutosten käsittelymekanismi vaikuttivat toteutuksen yksityiskohtiin.

Tilasäilöjen osalta käytössä olivat *useSharedValue* (React Native Reanimated) ja *mutableStateOf* (Kotlin Compose), joiden kautta tilamuuttujien päivitys tapahtui suoraan. Tämä mahdollisti lyhyen ja selkeän syntaksin kummassakin ympäristössä.

Merkittävin käytännön ero löytyi gesture-hallinnan toteutuksesta: React Nativessa (Reanimatedin avulla) *gesture handler* oli mahdollista kiinnittää suoraan siirrettävään elementtiin niin, että rotaation tunnistus toimi silti koko ruudun alueelta. Kotlin Multiplatform-toteutuksessa vastaavaa ratkaisua ei saatu toimimaan testiaikataulun puitteissa. Jotta siirtotoiminnon aloitus saatiin rajoittumaan ainoastaan elementin päälle, jouduttiin sen sijainti tunnistamaan erikseen ja konfiguroimaan ehdolliseksi.

Toisaalla elementin siirto saatiin toteutettua vähemmällä koodilla, Composen tarjotessa liikemuutoksen suoraan *positionChange*-funktion kautta (Kuva 17).



```
const deltaX = event.allTouches[0].absoluteX - event.allTouches[0].absoluteX;
const deltaY = event.allTouches[0].absoluteY - event.allTouches[0].absoluteY;
xPos.value = startX.value + deltaX;
yPos.value = startY.value + deltaY;
```

```
val firstFinger = activePointers.first()
offset += firstFinger.positionChange()
```

Kuva 17. Siirtomuutoksen koodi palapelinäkymässä Kotlin Multiplatformin ja React Nativen välillä.

Toisin kuin aiemmissa tämän luvun alaluvuissa, taulukkoon on tässä kohdassa sisällytetty enemmän koodirivejä, mikä antaa suuremman virhemarginaalin mahdollisten rivinvaihtojen tai nimeämispituuksien vaikutuksesta (Taulukko 8). Tällä kertaa kosketukseen liittyvien koodirivien erottelu oli kuitenkin abstraktimpaa, sillä suurin osa riveistä oli logiikkaa tai taustalla tapahtuvaa laskentaa. Tästä syystä nähtiin perustelluksi ilmoittaa koodirivit näin, vaikka tarkkuus ei ole täysin luotettavaa. Kokonaisuudessaan koodirivejä oli vähemmän Kotlin Multiplatform-toteutuksessa. Lopullisista arvoista on karsittu tyylilliset määrytykset sekä taustatoiminnot, joita ei ollut toteutettu vertailtavassa toisessa teknologiaratkaisussa.

Taulukko 8. Palapelinäkymän toteutunut koodirivimäärä.

Käytetyt kirjastot	Rivimäärä
Gesture Handler, Reanimated	177
Jetpack Compose	155

Dokumentaation ja kehittäjäkokemuksen vertailu

React Nativella oli tarjolla runsaasti dokumentaatiota, mikä tuki kehitystyötä merkittävästi. Kotlin Multiplatformin tapauksessa dokumentaatio ja esimerkit olivat sen sijaan varsin perustasoisia, eikä niistä ollut juurikaan apua vaativampien ratkaisujen toteutuksessa.

Molemmilla alustoilla muutosten tekeminen oli siinä mielessä intuitiivista, että suuri osa toiminnallisuudesta oli toteutettava itse, ja toteutustapa oli pitkälti vapaasti valittavissa. Tämä edellytti aiemmin tutuksi tulleen eleiden kuunteluun tarkoitetun rakenteen hyödyntämistä, jonka avulla kaikkien kosketustapahtumien seuranta oli mahdollista.

Kokonaisuutena React Native osoittautui palapelinäkymän toteutuksessa suoraviivaisemmaksi ja kehitykseltään nopeammaksi ympäristöksi. Reanimated tarjosi lisäksi erilliset *hooks* eleen aloitukselle, jatkuvalla päivittymiselle, sekä loppumiselle, mikä teki logiikasta helpommin hallittavaa ja luettavampaa.

Yllättävää oli, että kummankaan teknologian sisäänrakennetut rotaatioliikkeet eivät soveltuneet tähän käyttötapaukseen. Niiden rajoitteet edellyttivät täysin manuaalista toteutusta raakojen *pointer event*-proppien käsittelyn kautta. On kuitenkin huomioitava, että vastaavanlaiset palapeliratkaisut ovat tyypillisempiä peliympäristöissä, joissa kehitystyötä tukevat valmiit pelimootorit. Ne saattaisivat sisältää käyttäjäystävällisempiä ja vähemmän koodia vaativia vaihtoehtoja vastaavaan toiminnallisuuteen.

8 Kokonaisuuden vertailu tulosten pohjalta

Kokonaisuutena sekä React Native että Kotlin Multiplatform tarjosivat yllättävän tasavertaisen kokemuksen monimutkaisten kosketustoiminnallisuuden toteuttamisessa. Lopputulosten suorituskyky oli kummassakin teknologiassa samankaltainen, ja käyttäjäkokemuksen sulavuus kaikki tapahtumat huomioiden oli melko vastaavanlainen. React Nativessa erityisesti komponenttien selkeä segmentointi teki koodin rakenteesta helposti hahmotettavan ja helpotti kehitystyötä. Lisäksi React Nativelle tarjolla oleva laaja ja helposti lähestyttävä verkkodokumentaatio teki uuden oppimisesta sekä ongelmanratkaisusta nopeampaa ja sujuvampaa.

Kotlin Multiplatformin etuina korostuivat lopputulosten lyhyempi koodi ja sisäänrakennetut työkalut, jotka yksinkertaistivat esimerkiksi elementtien sijaintien vertailua ja muutosten hallintaa. Tämä mahdollisti nopeamman toteutuksen tietyissä tilanteissa. Haasteita kuitenkin ilmeni selvästi enemmän, ja osa niistä oli vaikeasti jäljitettävissä. Esimerkiksi rotaation toimimattomuus visuaalisesti tietyissä komponenttirakenteissa herätti epäselvyyksiä, sillä koodi ei itsessään näyttänyt olevan virheellinen. Dokumentaatio ei myöskään aina tukenut tehokasta ongelmanratkaisua, sillä se oli usein rakenteeltaan pirstaleista ja teknisiltä kuvauksiltaan tiivistettyä, minkä lisäksi tarjotut esimerkit olivat yksittäisiä ja yleistasoisia, eikä niistä usein löytynyt suoria vastauksia haettuihin kysymyksiin.

Teknologioiden välillä oli selviä eroja muun muassa syntaksissa, kuten Reactin JSX-rakenteissa ja hakasulkeissa, sekä sisäänrakennettujen funktioiden ja ominaisuuksien nimityksissä. Yllättävää oli, että kummassakaan teknologiassa rotaatioon suunnitellut sisäänrakennetut funktiot eivät tarjonneet riittävää hallintaa monikosketustoimintojen toteutukseen, vaan nämä oli rakennettava yleisemmillä eleentunnistimilla, jotka mahdollistivat tarvittavan toiminnallisuuden niiden tarjotessa kaikkien kosketusten sijainti- ja ominaistiedot.

Saumatonta kehityskokemusta ei saavutettu kummankaan teknologian kohdalla, ja tekoälyavusteisella työskentelyllä tarvittiin säännöllisesti

henkilökohtaista virheenselvitystä. React Nativessa laajempi ja selkeämpi dokumentaatio mahdollisesti edesauttoi tekoälyn tehokkuutta, kun taas Kotlin Multiplatformin osalta dokumentaation hajanaisuus saattoi heikentää tekoälyn hyödyllisyyttä etenkin tarkemmissa teknisissä kysymyksissä.

Logiikan toteuttaminen, tilamuuttujien hallinta ja käyttöliittymäelementtien rakentaminen sujuivat yllättävän samankaltaisesti molemmissa teknologioissa. Alkualetukseni oli, että kokemattomuuteni näiden kehitysalustojen erityispiirteissä johtaisi täysin erilaisiin toteutustapoihin, mutta käytännössä rakenteet tuntuivat molemmissa tutuilta ja johdonmukaisilta. Esimerkiksi viivojen piirtäminen tehtiin kummassakin erilliseen, dedikoituun komponenttiin, sijaintitiedot ja polut säilöttiin hyvin samankaltaisella tavalla. Erilaiset laskukaavat taas jouduttiin usein kirjoittamaan itse valmiiden ratkaisujen puuttuessa. Kun siirryttiin teknologiasta toiseen, työn aloittaminen tuntui huomattavasti kevyemmältä, sillä keskeiset konseptit olivat jo tuttuja ja niiden soveltaminen onnistui nopeasti.

Kaiken kaikkiaan React Native tuntui kehityskokemuksena suoraviivaisemmalta ja helpommin lähestyttävältä, erityisesti kun kyse oli useamman UI-elementin samanaikaisesta hallitsemisesta, virheenselvityksestä, ja dokumentaation etsinnästä. Kotlin Multiplatformissa puolestaan luottamusta herätti tapa, jolla käyttöliittymäelementtien sijainti laskettiin automaattisesti oikein omia laskukaavoja ja sijaintivertailuja varten. Lisäksi näiden elementtien sijainnin muuttaminen osoittautui joustavaksi ja vaivattomaksi, kun eletapahtumista saatiin suoraan uusi arvo yhdistettäväksi. Tämä antoi varmuutta siitä, että toteutus toimisi luotettavasti myös eri laitteilla, joiden erilaiset ruutukoot ja kiinteät näyttöalueet voisivat muutoin aiheuttaa vääristymiä tai odottamattomia poikkeamia.

9 Yhteenveto ja pohdinta

Opinnäytetyön edistyminen sujui tasaisesti koko prosessin ajan. Työskentelyä helpottivat aiempi kokemus ohjelmistokehityksestä ja ongelmanratkaisusta, sekä tekoälyn tehokas hyödyntäminen osana kehitystyötä. Vaikka työn aikana ilmeni haasteita, niiden ratkaiseminen sujui pääosin nopeasti, sillä ongelmanratkaisuun tarvittava ajattelutapa oli ennestään tuttu.

Eniten aikaa vei kehitysalustojen käyttöönotto sekä virheenkorjauksen toiminnan selvittäminen Android Studiossa. Koska nämä asiat eivät kuuluneet työn varsinaisen teoriaosuuden aihepiiriin, niitä ei ole käsitelty tekstissä tätä enempää.

Opinnäytetyöni aikana opin, millaisia rakenteita kosketustoiminnallisuuden toteutuksessa on otettava huomioon. Alkuperäisiin odotuksiini nähden näitä rakenteita oli vähemmän kuin olin ennakoanut, mikä madalsi kynnystä lähestyä kosketustoiminnallisuuden kehittämistä tulevilla projekteilla. Tämän kokemuksen myötä osaan jatkossa arvioida aiempaa realistisemmin, kuinka paljon aikaa erilaisten kosketustoiminnallisuuden toteuttaminen vaatii eri projektikonteksteissa.

Sain myös käytännön kokemusta Android Studion käytöstä ja opin, kuinka projektin voi peilata toimimaan fyysisellä Android-laitteella. Tämä on hyödyllinen taito, mikäli jatkan mobiilisovellusten kehittämistä Android Studion IDE-ympäristössä. Lisäksi opin käyttämään Kotlin-ohjelmointikieltä sekä sen tarjoamia toiminnallisuuden ominaisuuksia, erityisesti Kotlin Multiplatform -teknologian osalta. Näistä hyödylliseksi osoittautuivat esimerkiksi ratkaisut käyttöliittymäelementtien sijaintien hallintaan.

React Native -kehityksessä perehdyin eleiden käsittelyyn, mikä tarjoaa valmiuksia kosketustoiminnallisuuden toteuttamiseen myös kyseisellä alustalla. Tämä laajensi osaamistani erityisesti käyttöliittymien vuorovaikutteisuuden näkökulmasta.

Työskentelyn aikana havaittiin, että eri kirjastot tarjoavat vaihtelevia ratkaisuja kosketustoiminnallisuuksien toteuttamiseen. Tässä työssä käytetyt kirjastot helpottivat toteutusta merkittävästi, mutta niissä ilmeni myös puutteita.

Esimerkiksi osa toiminnallisuuksista vaati manuaalisia laskelmia, ja kirjastojen tarjoama joustavuus ei kaikilta osin vastannut haluttuja käyttötapauksia. On mahdollista, että jotkut muut kirjastot tarjoavat laajempia ja joustavampia ominaisuuksia, ja voivat näin yksinkertaistaa toteutusta.

Tämän opinnäytetyön tuloksia voidaan hyödyntää muodostettaessa käsitystä siitä, kuinka kosketustoiminnallisuuksien toteutus onnistuu kummallakin tarkastellulla kehityskehyksellä vaikeusasteen näkökulmasta sekä siitä, miten niiden toteutuksen monimutkaisuus eroaa keskenään. Lisäksi työ tarjoaa pohjustavan ohjeistuksen siitä, miten kosketustoiminnallisuuksien rakennetta voidaan lähteä rakentamaan molemmilla alustoilla. Samalla se tuo esiin havaintoja mahdollisista ongelmakohtista, joita kehitysprosessin aikana voi esiintyä.

Jatkokehityksen kannalta voitaisiin arvioida kosketustoiminnallisuuksien toteuttamista myös tällaisten vaihtoehtoisten kirjastojen avulla. Tämä mahdollistaisi kattavamman vertailun ja auttaisi löytämään tarkoituksenmukaisimman ratkaisun kuhunkin käyttötilanteeseen.

Lähteet

Akhin, M. & Belyaev, M. 2025. Kotlin language specification. Viitattu 24.1.2025. Kotlinlang.org-sivusto. <https://kotlinlang.org/spec/type-system.html>.

Ali, S. & Qayyum, S. 2021. A Pragmatic Comparison of Four Different Programming Languages. Viitattu 20.1.2025. ScienceOPEN.com-sivusto. <https://www.scienceopen.com/hosted-document?doi=10.14293/S21991006.1.SOR-.PP5RV1O.v1>.

Android Developers 2025a. Develop UI for Android | Jetpack Compose | Android Developers. Android Developers-sivusto. Viitattu 24.1.2025. <https://developer.android.com/develop/ui>

Android Developers 2025b. Graphics in Compose | Jetpack Compose | Android Developers. Viitattu 15.4.2025. <https://developer.android.com/develop/ui/compose/graphics/draw/overview>

Android Developers 2025c. Graphics modifiers | Jetpack Compose | Android Developers. Viitattu 12.4.2025. <https://developer.android.com/develop/ui/compose/graphics/draw/modifiers>.

Android Developers 2025d. Improve your code with lint checks | Android Studio | Android Developers. Android Developers-sivusto. Viitattu 24.1.2025. <https://developer.android.com/studio/write/lint>.

Android Developers 2025e. Jetpack Compose UI App Development Toolkit - Android Developers. Viitattu 15.4.2025. <https://developer.android.com/compose>.

Android Developers 2025f. Known issues with Android Studio and Android Gradle Plugin | Android Developers. Android Developers-sivusto. Viitattu 12.2.2025. <https://developer.android.com/studio/known-issues>.

Android Developers 2025g. Lifecycle of composables | Jetpack Compose | Android Developers. Viitattu 14.5.2025.
<https://developer.android.com/develop/ui/compose/lifecycle>.

Android Developers 2025h. Meet Android Studio | Android Developers. Android Developers-sivusto. Viitattu 12.1.2025.
<https://developer.android.com/studio/intro>.

Awati, R. 2024. What is a framework? | Definition from TechTarget. TechTarget sivusto. Viitattu 9.1.2024.
<https://www.techtarget.com/whatis/definition/framework>.

Cavalcante, A. 2019. What Is DX? (Developer Experience). Medium-sivusto. Viitattu 21.1.2025. <https://medium.com/swlh/what-is-dx-developer-experience-401a0e44a9d9>. Chrzanowska, N. 2024. React Native vs Swift - Performance and Development Comparison. Netguru-sivusto. Viitattu 15.1.2025.
<https://www.netguru.com/blog/swift-vs-react-native>.

Crudu, A. 2024. Designing for Multi-Touch Gestures in Mobile Applications. Viitattu 8.5.2025. <https://moldstud.com/articles/p-designing-for-multi-touch-gestures-in-mobile-applications>.

Purple Wren Digital 2023. Expo Updates. Medium-sivusto. Viitattu 10.1.2025.
https://medium.com/@caleb_23647/expo-updates-fa846ce96640.

Ducrohet, X.; Norbye, T. & Chou, K. 2013. Android Studio: An IDE built for Android. Android Developers blogi-sivusto. Viitattu 13.1.2025. <https://android-developers.googleblog.com/2013/05/android-studio-ide-built-for-android.html>.

Expo 2024a. Android Studio Emulator. Expo Docs-kirjasto. Viitattu 13.1.2025.
<https://docs.expo.dev/workflow/android-studio-emulator/>.

Expo 2024b. Development and production modes. Expo Docs-kirjasto. Viitattu 10.1.2025. <https://docs.expo.dev/workflow/development-mode/>. Liite 3 61

Expo 2025a. EAS Build Limitations. Expo Docs-kirjasto. Viitattu 12.2.2025.
<https://docs.expo.dev/build-reference/limitations/>.

Expo 2025b. Expo Go - Expo. Expo-sivusto. Viitattu 10.1.2025.

<https://expo.dev/go>. Expo 2025c. Expo Home page. Expo-sivusto. Viitattu 10.1.2025. <https://expo.dev/>.

Expo 2025d. Understanding app size. Expo Docs-kirjasto. Viitattu 12.2.2025.

<https://docs.expo.dev/distribution/app-size/>.

Flexible 2025. React Native vs Kotlin - A Detailed Comparison | Flexiple -

Flexiple. Flexible-sivusto. Viitattu 23.1.2025.

<https://flexiple.com/compare/react-native-vs-kotlin>.

Gesture Handler 2025. Rotation gesture | React Native Gesture Handler.

Viitattu 6.4.2025.

<https://docs.swmansion.com/react-native-gesture-handler/docs/gestures/rotation-gesture/>.

Heinonen, E. 2020. Ruby- ja Python-ohjelmointikielten vertailu.

Kandidaatintutkielma (YO). Tieto- ja sähkötekniikka. Tampere. Tampereen yliopisto. Tampere University-sivusto. Viitattu 20.1.2025.

<https://trepo.tuni.fi/handle/10024/123884>.

JetBrains 2024a. Kotlin Docs | Kotlin for server side. Kotlinin arkisto. Viitattu

16.1.2025. <https://kotlinlang.org/docs/server-overview.html>. JetBrains 2024b.

Kotlin Native | Kotlin. Kotlinin arkisto. Viitattu 17.1.2025.

<https://kotlinlang.org/docs/native-overview.html>.

JetBrains 2025. Kotlin Multiplatform Wizard. Viitattu 13.5.2025.

<https://kmp.jetbrains.com/>.

Kasle, M. 2024. "Mastering Code Quality in React Native: ESLint, Prettier, Commitlint, and Husky Setup for 2024". Medium-sivusto. Viitattu 24.1.2025.

Liite 3 62 <https://medium.com/%40manthankaslemk/mastering-code-quality-in-react-native-eslint-prettier-commitlint-and-husky-setup-for-2024-3ff7e47eca81>.

Katariya, L. 2023. React Native vs Kotlin: What Everything You Need To Know. Brilworks-sivusto. Viitattu 23.1.2025.

<https://www.brilworks.com/blog/react-native-vs-kotlin/>.

Ktlint n.d. Welcome to Ktlint. Pinterest-sivusto. Viitattu 24.1.2025.

<https://pinterest.github.io/ktlint/latest/>.

Lutkevich, B. 2024. What is Kotlin and Why Use it? | Definition from TechTarget. TectTarget-sivusto. Viitattu 17.1.2025.

<https://www.techtarget.com/whatis/definition/Kotlin>.

Maciej B. 2024. What Is React Native? Complex Guide for 2024. Netguru sivusto. Viitattu 15.1.2025. <https://www.netguru.com/glossary/react-native>.

Miller, N. 2020. Stylus vs Finger: Which Touchscreen Input Is Best?. Viitattu 8.5.2025. <https://nelson-miller.com/stylus-vs-finger-which-touchscreen-input-is-best/>.

Nyrhinen, M. 2022. JavaScriptin ja Pythonin vertailu web-kehityksessä. Kandidaatintutkielma (YO). Tieto- ja sähkötekniikka. Tampere. Tampereen yliopisto. Tampere University-sivusto. Viitattu 20.1.2025.

<https://trepo.tuni.fi/handle/10024/141773>.

OpenAI 2025a. ChatGPT (Helmikuu GPT-4-turbo versio). Viitattu 12.2.2025.

<https://chat.openai.com/chat>.

Pereira, R.; Couto, M.; Ribeiro, F.; Rua, R.; Cunha, J.; Fernandes, J. & Saraiva, J. 2017. Energy efficiency across programming languages: How do energy, time, and memory relate?. Viitattu 20.1.2025. SLE 2017 - Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Liite 3 63 Engineering, co-located with SPLASH 2017. Association for Computing Machinery, Inc, 256–267. <https://dl.acm.org/doi/10.1145/3136014.3136031>.

Polus, P. 2024. React Native vs Kotlin Multiplatform Mobile | Blog Miquido. Miquido-sivusto. Viitattu 23.1.2025. <https://www.miquido.com/blog/kotlin-multiplatform-vs-react-native/>.

React Native 2025a. Core Components and APIs · React Native. React Native kirjasto. Viitattu 13.1.2025. <https://reactnative.dev/docs/components-and-apis>.

React Native 2025b. React Native · Learn once, write anywhere. React Native sivusto. Viitattu 13.1.2025. <https://reactnative.dev/>.

React Native Skia 2025. React Native Skia | React Native Skia, Viitattu 15.4.2025. <https://shopify.github.io/react-native-skia/>.

Reanimated 2025. React Native Reanimated. Viitattu 15.4.2025. <https://docs.swmansion.com/react-native-reanimated/>.

Reddit n.d. What are the disadvantages of the Expo?. Reddit-sivusto. Viitattu 12.2.2025. https://www.reddit.com/r/reactnative/comments/f0a8zh/what_are_the_disadvantages_of_the_expo/.

Sipola, M. 2023. Ohjelmointikielten vertailu web-ohjelmoinnissa. Opinnäytetyö (AMK). Tieto- ja viestintätekniikka. Jyväskylä. Jyväskylän ammattikorkeakoulu. Theseus-sivusto. Viitattu 21.1.2025. https://www.theseus.fi/bitstream/handle/10024/809631/Opinnaytetyo_Sipola_Mikko.pdf.

Software Mansion n.d. Introduction | React Native Gesture Handler. React Native Gesture Handler Docs. Viitattu 22.2.2025. <https://docs.swmansion.com/react-native-gesture-handler/docs/>.

StackOverflow 2019. Does React Native have a "Virtual DOM"? StackOverflow sivusto. Viitattu 24.1.2025. <https://stackoverflow.com/questions/41804855/does-react-native-have-a-virtual-dom>. Liite 3 64.

Steele, C. & Darling, E. 2023. What is mobile application development platform (MADP)? | Definition from TechTarget. TechTarget-sivusto. Viitattu 10.1.2025. <https://www.techtarget.com/searchmobilecomputing/definition/mobile-application-development-platform>.

Ylisirniö, J. 2024. Mobiilisovelluksen kehittäminen osana tuotekehitystä. Opinnäytetyö (AMK). Tieto- ja viestintätekniikka. Tampere. Tampereen ammattikorkeakoulu. Theseus-sivusto. Viitattu 9.1.2025.
https://www.theseus.fi/bitstream/handle/10024/873880/Ylisirnio_Jenni.pdf.